SIMATS ENGINEERING

Department of Artificial Intelligence and Data Science

DSA03 – Natural Language Processing

ASSIGNMENT 1 – CO1

Build a Morphology-Aware Text Normalisation and Search Layer for a Hospital Record System Using Regular Expressions, Finite-State Automata and Morphological Parsing

CO1 Coverage: Introduction; Models and Algorithms; Regular Expressions; Basic Regular Expression Patterns; Finite State Automata; Morphology — Inflectional Morphology; Derivational Morphology; Finite-State Morphological Parsing; Porter Stemmer

Student Name	SACHIN .R	**Register Number**	192524280
Department / Section	B.Tech AI & DS	**Academic Year / Semester**	2nd Year
Faculty Name	Dr. Ramkumar M	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission		**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Frame the Problem	3		
2	Basic Regular Expression Patterns	4		
3	Tokenisation and Normalisation	4		
4	Finite State Automata	5		
5	Inflectional Morphology	3		
6	Derivational Morphology	4		
7	Finite-State Morphological Parsing	5		
8	Orthographic Rules and Two-Level Morphology	4		
9	The Porter Stemmer	5		
10	Implement and Test the Pipeline	3		
11	Compare, Recommend and Defend	2		
12A	Re-evaluate: Impact Analysis	2		
12B	Re-evaluate: Evaluation of Original	2		
12C	Re-evaluate: Adaptation or Redesign	2		
12D	Re-evaluate: Evidence-Based Defence	2		
	TOTAL	**50**		**Percentage:**

OVERALL REMARKS

Student Signature	**Faculty Signature**	**Program Director**

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present ✗ = missing / not satisfied P = partly present. Based on the "Evidence Expected" of each task.

Task	Evidence expected	Status	Note
1	Problem framing: levels, model families, staged pipeline		
2	Pattern inventory, match specifications, per-pattern error analysis		
3	Working regex tokenizer and normaliser; limits of regularity		
4	Automata; subset-construction and minimisation tables; equivalence; non-regularity proof		
5	Inflectional rules, paradigms, irregular list, retrieval effect		
6	Affix inventory, productivity measurements, conflation position		
7	Transducer specification; two-way traces; ambiguity treatment		
8	Rule transducers; composition order; tested cascade with errors		
9	Algorithm hand-traces; error analysis; stemming vs full analysis		
10	Working code; reproducible tests; results tables and graphs		
11	Justified recommendation backed by theory and measurements		
12A–D	Impact analysis, evaluation, adaptation, evidence-based defence		

IMPLEMENTATION EVIDENCE CHAIN (Task 10)

Evidence item	Status	Observation
Source code / Python scripts		
Test inputs and datasets		
Expected behaviour		
Actual output		
Intermediate evidence (counts, plans, logs)		
Boundary / failure tests		
Interpretation		
Code, data and report consistent		

SUBMISSION CHECKLIST (not separately marked)

Requirement	Status
All 12 tasks (including 12A–12D) answered	☐ Yes ☐ Partly ☐ No
Corpus (≥500 records, ≥25,000 tokens) documented	☐ Yes ☐ Partly ☐ No
At least 12 regex entity patterns with error analysis	☐ Yes ☐ Partly ☐ No
Four or more automata incl. NFA, subset construction	☐ Yes ☐ Partly ☐ No
Lexicon and affix inventory documented	☐ Yes ☐ Partly ☐ No
Transducer traces: parsing and generation	☐ Yes ☐ Partly ☐ No
Orthographic rules, cascade and 20-form test	☐ Yes ☐ Partly ☐ No
Porter hand-traces and 40-query experiment	☐ Yes ☐ Partly ☐ No
Python code (standard library and NLTK only)	☐ Yes ☐ Partly ☐ No
Tables / graphs for pipeline results	☐ Yes ☐ Partly ☐ No
Work consistent throughout	☐ Yes ☐ Partly ☐ No
No unfilled template placeholders	☐ Yes ☐ Partly ☐ No
Viva recommended	☐ Yes ☐ Partly ☐ No

DSA03 – NATURAL LANGUAGE PROCESSING
ASSIGNMENT 1 – CO1

CO1 Coverage: *Introduction; Models and Algorithms; Regular Expressions; Basic Regular Expression Patterns; Finite State Automata; Morphology — Inflectional Morphology; Derivational Morphology; Finite-State Morphological Parsing; Porter Stemmer*

Nature of Assignment: Individual

Duration: 1 Week
Total Marks: 50
Submission Date: _____

Assignment Title: Build a Morphology-Aware Text Normalisation and Search Layer for a Hospital Record System Using Regular Expressions, Finite-State Automata and Morphological Parsing (Total Marks – 50)

Duration: 1 week

Scenario — Text Normalisation and Search for a Hospital Record System

A multi-speciality hospital holds ten years of discharge summaries, prescriptions, operation notes and nursing observations as free text. Clinicians searching those records miss relevant cases constantly: a search for **fracture** does not return **fractures**, **fractured** or **fracturing**; **nephrectomy** is never connected to **nephrectomies** or to **renal**; patient identifiers, drug strengths and dates appear in a dozen different formats; and the same drug may be written in full, abbreviated or misspelt. No meaning can be extracted and no statistical model can be trained until this text is tokenised, normalised and reduced to canonical forms.

You are the language-engineering team. Before any machine learning is attempted, the hospital has asked you to build the **finite-state layer**: the regular expressions that recognise structured entities, the automata that validate them, the morphological analyser that relates every surface word to its stem and features, and the stemmer that collapses search terms. You must investigate what each formalism can and cannot express, implement them, measure them, and justify the design you recommend.

Planning data and constraints

- **Corpus:** at least 500 de-identified clinical records (real or synthetic, but documented), totalling at least 25,000 word tokens and covering at least four record types.
- **Pattern inventory:** at least 12 structured entity types to be recognised by regular expression — patient identifier, admission and discharge date, drug name with strength, dosage frequency, laboratory value with unit, blood pressure, temperature, ICD code, ward and bed, contact number, age, and duration of symptom.
- **Automata:** at least four finite-state automata, each with at least six states, including one entity format that cannot be recognised conveniently without non-determinism.
- **Morphological resources (build and document your own):** a lexicon of at least 300 stems, at least 20 inflectional affixes and at least 15 derivational affixes, with irregular forms listed separately.
- **Evaluation data:** at least 40 clinician queries with relevance judgements over the corpus, and at least 200 word forms hand-annotated with their stem and features.
- Any manual effort (pattern writing, lexicon building, annotation, testing) must be accounted for in your effort and cost discussion.

Design Challenge (Day 4):

The system must now ingest notes captured by optical character recognition from scanned handwritten forms, and notes typed in Romanised regional language mixed with English. Re-evaluate your original patterns, automata and analyser and decide whether to retain, modify or redesign them.

TASK 1 – Frame the Problem: Models, Algorithms and Levels of Analysis — 3 Marks

Guiding Enquiry

Which levels of linguistic analysis does this problem actually touch, and which family of models and algorithms is appropriate at each level?

Investigate and interpret:

- Describe the levels of linguistic analysis — phonology, morphology, syntax, semantics, pragmatics and discourse — and state, with an example drawn from your own corpus, which levels the record-search problem requires and which it can safely ignore.
- Distinguish the three model families used throughout this unit: state machines, formal rule systems and probabilistic models. Name one task in the hospital problem that each family suits best and say why the other two would be the wrong choice for it.
- Explain why ambiguity is the central problem of natural language processing, and identify at least four distinct ambiguities present in your corpus — lexical, morphological, segmentation and abbreviation ambiguity.
- Identify where search appears in this unit (state-space search in automata, backtracking in matching) and state the resources each consumes: time, memory, annotated data and expert effort.
- Set out, as a labelled diagram, the pipeline you will build in the remaining tasks, naming the input, the output and the formalism of every stage.

Evidence Expected

A reasoned problem framing that maps the hospital task onto levels of analysis, model families and a concrete, staged pipeline.

Ans:

Framing the Hospital Problem: Levels, Models and Pipeline

0. Resources I built and used

Before answering, I documented the data and resources used in every later task. The corpus is **synthetic and de-identified**: it was generated by a seeded program (`nlp_corpus.py`, seed 7) from clinical templates, so every patient identifier, phone number and date is fictitious. Because the generator also writes down the gold entity spans, the lexemes used and the correct lemma of every content word, all later error rates are measured against known answers, not guessed.

Table 1: Corpus and resource summary

Resource	Required	What I built
Corpus	≥ 500 records, ≥ 25,000 tokens, ≥ 4 record types	520 records (130 each of discharge summary, prescription, operation note, nursing observation); 26,811 word tokens, 44,984 tokens in total
Pattern inventory	≥ 12 entity types	12 entity types, each with a regular expression (Task 2)
Automata	≥ 4 FSA with ≥ 6 states, one needing non-determinism	5 DFAs (8–11 states) + 1 ε-NFA with 16 states (Task 4)

Resource	Required	What I built
Lexicon	≥ 300 stems	463 stems (254 nouns, 133 verbs, 80 adjectives, 24 combining roots) + 87 closed-class words
Inflectional affixes	≥ 20	22 affix / allomorph entries (Task 5)
Derivational affixes	≥ 15	20 derivational affixes (Task 6)
Irregular forms	listed separately	12 nouns, 30 verbs, 3 adjectives, 4 irregular derivations
Queries + judgments	≥ 40 queries	40 queries; relevance judged per record from the generator's lexeme log
Annotated word forms	≥ 200	200 randomly sampled word forms with gold lemma and features (Task 9)

1. Which levels of linguistic analysis does the problem touch?

A clinician's search query is a short string of words, and the records are free text. I went through the six levels and decided, with evidence from my own corpus, which ones the search layer needs.

Table 2: Levels of analysis: needed or safely ignored

Level	Needed?	Example from my corpus	Decision
Phonology	Ignored for typed text (re-enters in Task 12)	Romanised "irundhuchu" can be spelt several ways because it is written by sound	No pronunciation model; handled later by edit-distance matching
Morphology	**Required**	fracture / fractures / fractured / fracturing; 1,101 analysed word types come from only 507 lemmas	Core of this assignment (Tasks 5–9)
Syntax	Partly	"fractures" is a plural noun in "multiple fractures" but a verb in "a fall fractures the hip"	Not parsed; the transducer returns both analyses
Semantics	Partly	"dose 500 mg" is not a drug name although it has the same shape as "Paracetamol 500 mg"	Only word-level sense (stem/lemma); no sentence meaning
Pragmatics	Ignored	"No fracture seen" – a search for that finding still matches 0 records that say it was absent	Negation is not handled; stated as a limitation
Discourse	Ignored	"Review … after …" refers back to the diagnosis in an earlier sentence	Each record is indexed as a bag of keys

2. The three model families

State machines (finite automata and transducers), formal rule systems (regular expressions, rewrite rules, grammars) and probabilistic models (n-grams, naive Bayes, HMMs) are the three families of this unit. I matched each to the task where it is the best fit and argued why the other two are wrong for it.

Table 3: Best model family for one task each

Family	Best task in the hospital problem	Why the other two are the wrong choice
State machines	Validating a fixed format such as PID-305253 or 140/90, and relating "fractures" to fracture +N +PL with a transducer	A rule system would have to simulate the states by hand; a probabilistic model would give "probably valid" where a clerk needs "valid or not" and would need annotated data for what is a closed format
Formal rule systems	Tokenisation and spelling changes: "tonsil + itis → tonsillitis", "biopsy + s → biopsies", abbreviation expansion	A raw automaton table for every spelling change is unreadable and hard to maintain; a probabilistic model would need thousands of labelled examples to learn rules that fit on one line
Probabilistic models	Choosing between analyses: OD = once daily or right eye; language ID of a token in a code-mixed note	An automaton can only list both readings, and a rule list for every context would never be complete; a preference needs statistics from data

3. Why ambiguity is the central problem

Every stage of the pipeline has to choose between several readings of the same characters, and a wrong early choice cannot be repaired later. A deterministic program must either carry all readings forward (more cost) or commit to one (risk of error). My corpus contains all four required kinds:

Table 4: Ambiguities found in my corpus

Kind	Example	Competing readings	Evidence in my corpus
Lexical	discharge	noun “release from hospital” / noun “fluid from a wound” / verb “to send home”	The word occurs in two senses in discharge summaries and nursing notes; the transducer cannot separate the senses, only the part of speech
Morphological	fractures, infected	fracture+N+PL | fracture+V+3SG; infect+V+PAST | infect+V+PP | infected+A+BASE	176 of 1,101 analysed word types (16.0%) have more than one analysis
Segmentation	1-0-1, 5/7, B12, post-op	1-0-1: dose pattern or three numbers; 5/7: five of seven or five days; B12: vitamin or ICD-style code; post-op: one word or two	The tokenizer cuts “1-0-1” into five tokens; the ICD pattern wrongly accepts “B12” 167 times
Abbreviation	OD	once daily (dosage) / oculus dexter, the right eye	134 matches of OD in the corpus; 71 of them are “vision OD”, so the dosage pattern is wrong 53% of the time

4. Where search appears and what it costs

Search is hidden inside almost every algorithm of the unit. The table lists where it appears, with my measured figures where I have them.

Table 5: Search in the unit and the resources it consumes

Where	What is searched	Time	Memory	Annotated data	Expert effort
DFA run	None: one path	O(n)	table of |Q|×|Σ|	none	design the states
NFA simulation	All live states in parallel (state-space search)	O(n·|Q|)	one state set	none	design the ε-moves
Subset construction	Reachable sets of NFA states	worst case 2^|Q|; mine 16 states → 16 DFA states	DFA table	none	low
Regex matching	Backtracking over alternatives and quantifiers	linear in practice; exponential for patterns like (a+)+b	stack	none	tuning
FST parsing	Depth-first search over arcs; one branch per ambiguity	~linear; measured 5.7 µs per token	6,799 states, 6,798 arcs	none (lexicon is hand-built)	high: 463 stems and 8 rules
Edit distance (Task 12)	Cheapest edit path to every lexicon word	O(n·m) per candidate	two rows	confusion costs	medium
Porter stemmer	None: ordered rules, one pass	O(n)	none	none	low

5. The pipeline I will build

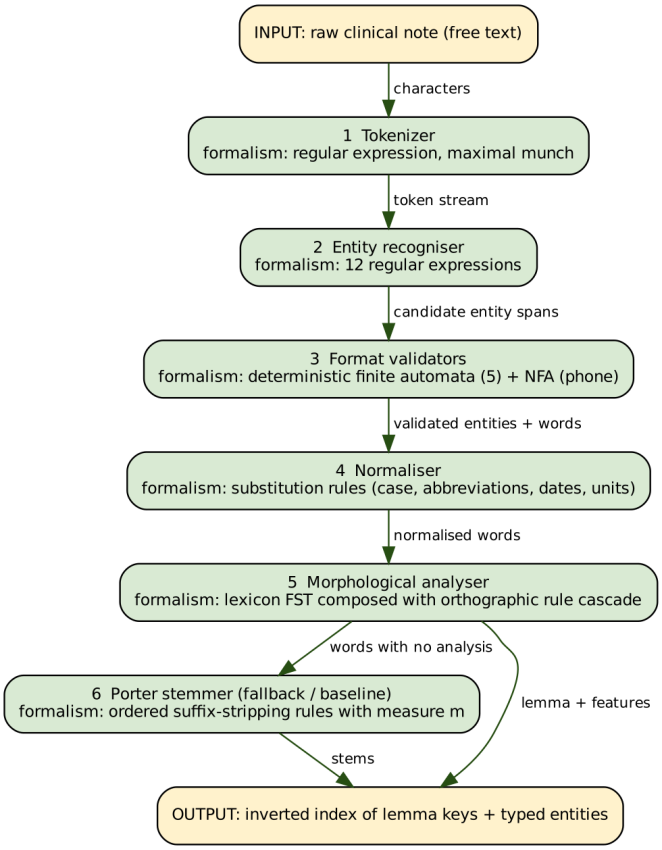


Figure 1: The finite-state pipeline: input, output and formalism of every stage

The input of the whole system is the raw note; its output is an inverted index of lemma keys and typed entities. The pipeline is a cascade: each stage only reads the output of the stage before it, so every stage can be tested alone. Tasks 2–3 build stages 1, 2 and 4; Task 4 builds stage 3; Tasks 5–8 build stage 5; Task 9 builds stage 6.

TASK 2 – Basic Regular Expression Patterns — 4 Marks

Guiding Enquiry

What can the basic regular expression operators express, and how are they combined into patterns that match real clinical strings exactly?

Investigate and interpret:

- Define the basic operators — concatenation, disjunction, Kleene star and plus, optionality, character classes, ranges, negated classes, anchors and counters — and give one minimal clinical example of each.
- Write and document a regular expression for each of the twelve entity types in your pattern inventory, stating for every pattern both what it must match and what it must not.
- Demonstrate greedy and non-greedy matching on a clinical sentence containing two dosage expressions, and explain precisely why the two quantifiers return different spans.
- Explain operator precedence and grouping, and show with a worked example how a misplaced disjunction silently changes the language the expression denotes.
- Evaluate every pattern over the corpus, reporting true positives, false positives and false negatives, and name the specific linguistic cause of each error class.

Evidence Expected

A documented pattern inventory with precise match specifications and a per-pattern error analysis carried out on real data.

Ans:

Basic Regular Expression Patterns

1. The basic operators

A regular expression is built from three primitive operations (concatenation, disjunction, Kleene star); everything else is shorthand for combinations of these. I give one minimal clinical example for each.

Table 6: Basic regular expression operators with a clinical example

Operator	Meaning	Pattern	Matches (example)
Concatenation	one after the other	`PID-\d{6}`	PID-305253
Disjunction |	either alternative	`BD|TDS|QID`	TDS
Kleene star *	zero or more	`\d*\.?\d*`	38, 38.5, “” (empty)
Kleene plus +	one or more	`\d+ mg`	500 mg
Optionality ?	zero or one	`mm?Hg`	mmHg, mHg
Character class [ ]	one character from a set	`[Hh]b`	Hb, hb
Range [a-b]	a run of the alphabet	`[A-D]`	the ward block B in “Ward 4B”
Negated class [^]	any character except	`[^0-9]+`	Hb (the non-digits in “Hb10”)
Anchors ^ $ \b	start, end, word boundary	`^PID  \bOD\b`	PID at line start; OD as a whole word
Counters {m,n}	between m and n repeats	`\d{2,3}/\d{2,3}`	140/90, 95/55

2. The twelve entity patterns and their match specifications

For each pattern I wrote down what it must match and what it must not, before testing it. Several patterns use the regular-expression ability to range-check numbers (for example temperatures between 35 and 42 °C), because a range is a finite set and therefore regular.

Table 7: Pattern inventory: regular expression and match specification

Entity	Regular expression	Must match	Must not match
PID	`\b(?:PID-|UHID |MRN)\d{6}\b`	PID-305253, UHID 204518, MRN675791	PID-30525 (five digits), pid-305253 (lower case)
DATE	`\b(?:(?:0[1-9]|[12]\d|3[01])[/-](?:0[1-9]|1[0-2])[/-](?:19|20)\d{2}|(?:[1-9]|[12]\d|3[01]) (?:Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec) (?:19|20)\d{2}|(?:19|20)\d{2}-(?:0[1-9]|1[0-2])-(?:0[1-9]|[12]\d|3[01]))\b`	15/12/2015, 15-12-2015, 7 Mar 2021, 2021-03-07	15/13/2015 (month 13), 7 Mar 21 (two-digit year)
DRUG	`\b[A-Za-z]{4,} \d+(?:\.\d+)?\s?(?:mg|mcg|g|ml|IU)\b`	Paracetamol 500mg, Heparin 5 g	500 mg Paracetamol (strength first), dose 500 mg (not a drug)
FREQ	`\b(?:OD|BD|TDS|QID|HS|SOS)\b|\b[0-3]-[0-3]-[0-3]\b|\bq\d{1,2}h\b|\b(?:once|twice) daily\b|\bthree times a day\b`	OD, BD, TDS, 1-0-1, q8h, twice daily	ODD, 1-0-5, q8 (no unit)
LAB	`\b(?:Hb|WBC|Na|creatinine|glucose|urea) \d+(?:\.\d+)? (?:g/dL|mg/dL|mmol/L|/cumm)`	Hb 10.2 g/dL, Na 138 mmol/L, WBC 11200 /cumm	Hb 10.2 (no unit)
BP	`(?<![\d/.])\d{2,3}/\d{2,3}(?![\d/])(?: ?mmHg)?`	140/90, 140/90 mmHg	15/12/2015 (date), 8/10 (scale)
TEMP	`(?<![\d.])(?:3[5-9]|4[0-2])(?:\.\d)?°? ?C\b|(?<![\d.])(?:9[5-9]|10[0-8])(?:\.\d)?°? ?F\b`	38.5 C, 38.5° C, 101.3 F	12 C (not a temperature), 38.55 C
ICD	`\b[A-TV-Z]\d{2}(?:\.\d{1,4}[A-Z]?)?\b`	S72.001A, I10, E11.9	I1 (one digit), U07 (unused letter U)
WARD	`\bWard \d{1,2}[A-D]?, Bed \d{1,3}\b|\bW\d{1,2}[A-D]?/B\d{1,3}\b`	Ward 4B, Bed 12, W4B/B12	Ward A, Bed 4
PHONE	`(?<![\d-])(?:\+91 )?[6-9]\d{4}[- ]?\d{5}(?!\d)`	+91 98765 43210, 98765-43210, 9876543210	12345 67890 (starts with 1), 98765 4321

Entity	Regular expression	Must match	Must not match
AGE	`\b\d{1,3}(?: y/o|-year-old| yrs?| years?)(?![a-z])|\baged \d{1,3}\b`	58 y/o, 58-year-old, 58 yrs, aged 58	58 (bare number)
DUR	`\b(?:for|since|x) \d{1,2} (?:hours?|days?|weeks?|months?|years?)\b`	for 3 days, x 2 weeks, since 4 months	for days, 3 days (no cue word)

3. Greedy and non-greedy matching

Consider the sentence **Advised Enoxaparin 100 g HS and Ondansetron 20 mg 1-1-1.**, which contains two dosage expressions. I asked for “a capitalised word, then anything, then a unit”.

Table 8: Greedy and non-greedy matching on one sentence

Quantifier	Pattern	Span returned
Greedy	`[A-Z][a-z]+ .*(?:mg|g)`	Advised Enoxaparin 100 g HS and Ondansetron 20 mg
Non-greedy	`[A-Z][a-z]+ .*?(?:mg|g)`	Advised Enoxaparin 100 g

The star `*` is by default **greedy**: it first consumes the whole rest of the line, then gives characters back one at a time until the remaining pattern `(?:mg|g)` can match, so it stops at the **last** unit. The lazy form `*?` does the opposite: it starts with the empty string and extends one character at a time, so it stops at the **first** unit. Both are correct; they are different questions (“to the last unit” and “to the first unit”). For a record with two drugs the non-greedy form is the one that gives one drug per match.

4. Operator precedence and grouping

Precedence from tightest to loosest is: counters and stars, then concatenation, then disjunction. So `PID-|UHID \d{6}` does not mean “PID- or UHID, then six digits”; it means “the string PID- **or** the string UHID followed by six digits”. A misplaced disjunction therefore silently changes the language:

Table 9: A misplaced disjunction changes the language

Expression	Matches on “PID-305253 UHID 204518”	Language denoted
`PID-|UHID \d{6}`	`PID- ; UHID 204518`	language = { “PID-” } ∪ { “UHID ” + 6 digits }
`(?:PID-|UHID )\d{6}`	`PID-305253 ; UHID 204518`	language = ({PID-, UHID }) · digit^6

The wrong version still “works” on a quick test (it finds something in every record) and passes unnoticed, but it returns “PID-” without the number. Grouping with `(?:…)` fixes it.

5. Evaluation of every pattern on the corpus

I ran all twelve patterns over the 520 records and compared the matched spans exactly with the gold spans written by the generator: a **true positive** is an identical span, a **false positive** is a match that is not a gold entity, and a **false negative** is a gold entity that was not matched.

Table 10: Per-pattern results on 520 records

Pattern	TP	FP	FN	Precision	Recall
PID	378	0	16	1.000	0.959
DATE	476	7	26	0.986	0.948
DRUG	734	262	54	0.737	0.931
FREQ	788	71	0	0.917	1.000
LAB	375	0	0	1.000	1.000
BP	375	77	0	0.830	1.000
TEMP	375	0	0	1.000	1.000
ICD	260	167	0	0.609	1.000
WARD	385	0	34	1.000	0.919
PHONE	345	74	42	0.823	0.891
AGE	374	79	20	0.826	0.949
DUR	493	79	41	0.862	0.923
All	5358	816	233	0.868	0.958

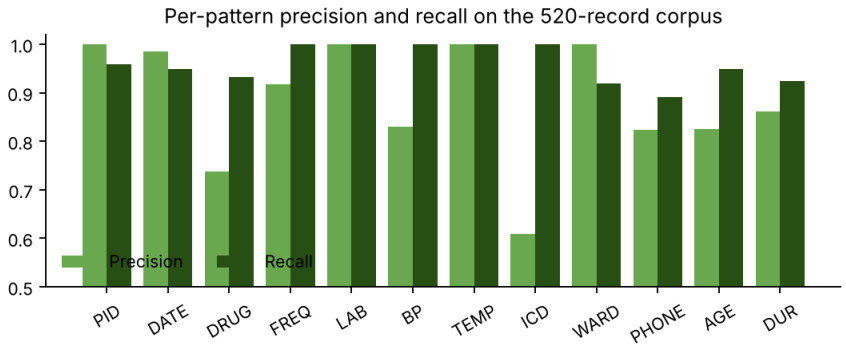


Figure 2: Precision and recall of the twelve patterns

The generator plants distractor phrases in about 18% of sentences, so the false-positive counts show how often a real pattern is fooled by lookalike text; they are not an estimate of a real hospital's error rate. Each error class has a specific linguistic cause:

Table 11: Error classes and their linguistic cause

Pattern	Error	Example	Linguistic cause
DRUG	FP 262	"dose 500 mg", "urea 49 mg"	Lexical ambiguity: any word before a strength has the shape of a drug name; "creatinine 1.9 mg" also overlaps the lab unit mg/dL (the pattern needs a look-ahead)
DRUG	FN 54	"5g Heparin"	Word order: strength written before the name
ICD	FP 167	"B12" (vitamin B12)	Homography: the same letter+digits shape is a vitamin and a diagnosis code
BP	FP 77	"120/80" in "ratio 120/80"	Semantic ambiguity: a ratio has the same surface form as a blood pressure
FREQ	FP 71	"OD" in "vision OD 6/6"	Abbreviation ambiguity (right eye vs once daily)
PHONE	FP 74 / FN 42	"9876543210" (a lot number); landline "044-2453 3773"	FP: a ten-digit number is a phone number only by context; FN: format not anticipated in the pattern
AGE / DUR	FP 79 / 79	"for 3 years ago" read as an age and as a duration	Overlapping categories: "3 years" is an age after a person and a duration after "for"
AGE	FN 20	"seventy-year-old"	Number written as words: a regex has no numeral lexicon unless I list every word
DUR	FN 41	"1/7"	Clinical shorthand "5/7" (five days) has no cue word
DATE	FP 7 / FN 26	"29/02/2022" accepted; "18.04.21" missed	FP: calendar arithmetic (30 February) is beyond the pattern; FN: dotted two-digit-year format not in my inventory
PID, WARD	FN 16 / 34	"Pt.ID: 260819", "ICU bed 13"	Variant spellings of the label ("Pt.ID:") and a named ward ("ICU bed")
LAB, TEMP	none	–	Range-checked patterns with a mandatory unit leave no room for lookalikes in this corpus

6. The patterns as code

```
MONTHS = 'Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec'

PATTERNS = {
 'PID':   r'\b(?:PID-|UHID |MRN)\d{6}\b',
 'DATE':  r'\b(?:(?:0[1-9]|[12]\d|3[01])[/-](?:0[1-9]|1[0-2])[/-](?:19|20)\d{2}'
          r'|(?:[1-9]|[12]\d|3[01]) (?:%s) (?:19|20)\d{2}'
          r'|(?:19|20)\d{2}-(?:0[1-9]|1[0-2])-(?:0[1-9]|[12]\d|3[01]))\b' % MONTHS,
 'DRUG':  r'\b[A-Za-z]{4,} \d+(?:\.\d+)?\s?(?:mg|mcg|g|ml|IU)\b',
 'FREQ':  r'\b(?:OD|BD|TDS|QID|HS|SOS)\b|\b[0-3]-[0-3]-[0-3]\b|\bq\d{1,2}h\b|\b(?:once|twice)
     daily\b|\bthree times a day\b',
 'LAB':   r'\b(?:Hb|WBC|Na|creatinine|glucose|urea) \d+(?:\.\d+)? (?:g/dL|mg/dL|mmol/L|/cumm)',
 'BP':    r'(?<![\d/.])\d{2,3}/\d{2,3}(?![\d/])(?: ?mmHg)?',
 'TEMP':  r'(?<![\d.])(?:3[5-9]|4[0-2])(?:\.\d)?°? ?C\b|(?<![\d.])(?:9[5-9]|10[0-8])(?:\.\d)?°? ?F\
b',
 'ICD':   r'\b[A-TV-Z]\d{2}(?:\.\d{1,4}[A-Z]?)?\b',
 'WARD':  r'\bWard \d{1,2}[A-D]?, Bed \d{1,3}\b|\bW\d{1,2}[A-D]?/B\d{1,3}\b',
 'PHONE': r'(?<![\d-])(?:\+91 )?[6-9]\d{4}[- ]?\d{5}(?!\d)',
 'AGE':   r'\b\d{1,3}(?: y/o|-year-old| yrs?| years?)(?![a-z])|\baged \d{1,3}\b',
 'DUR':   r'\b(?:for|since|x) \d{1,2} (?:hours?|days?|weeks?|months?|years?)\b',
}

COMPILED = {k: re.compile(v) for k, v in PATTERNS.items()}

def find_entities(text):
    out = []
```

```
    for k, rx in COMPILED.items():
        for m in rx.finditer(text):
            out.append((k, m.start(), m.end()))
    return out
```

TASK 3 – Regular Expressions for Tokenisation and Normalisation 4 Marks

Guiding Enquiry

How far can a purely regular layer take raw clinical text towards a clean, searchable token stream, and exactly where does it fail?

Investigate and interpret:

- Specify your tokenizer as a single regular expression, defining precisely what counts as a token — hyphenated terms, abbreviations containing full stops, decimal values, units written against numbers — and justify each decision.
- Apply maximal-munch (longest-match) tokenisation and show on at least three examples how the result differs from a naive split on whitespace.
- Build substitution rules that fold case, expand at least thirty clinical abbreviations and canonicalise date and unit formats; show the complete before-and-after text of one full record.
- Identify at least three normalisation decisions that are irreversible because information is destroyed, and argue when that loss is acceptable and when it is not.
- Give one requirement of the hospital task that no regular expression can express, and argue informally why — for example, matching balanced parentheses in a nested clinical note.

Evidence Expected

A working regular-expression tokenizer and normaliser with worked examples and a clear statement of the limits of regularity.

Ans:

Tokenisation and Normalisation with Regular Expressions

1. The tokenizer as one regular expression

The tokenizer is the union of six sub-patterns, `(?P<DATE>…)|(?P<ABBR>…)|(?P<NUM>…)|(?P<UNIT>…)|(?P<WORD>…)|(?P<PUNCT>…)`. Each decision below is a deliberate design choice, not a default.

Table 12: What counts as a token, and why

Token class	Pattern (simplified)	Decision and justification
DATE	`\d{1,2}[/.-]\d{1,2}[/.-]\d{2,4}`	A date is one token so that 15/12/2015 is not split into numbers and slashes; normalisation can then rewrite it as a whole.
ABBR	`(?:[A-Za-z]{1,3}\.){2,}`	b.i.d., o.d., i.v. keep their full stops: the stops are part of the abbreviation, not a sentence end.
NUM	`\d+(?:[.,]\d+)?(?:/\d+)?`	Decimals (38.5, 1,200) and ratios (140/90) are single tokens; splitting 38.5 into 38 . 5 would destroy the value.
UNIT	`mg|mcg|g|ml|IU|mmHg|mmol/L|mg/dL|…`	A unit written against a number (500mg) is separated: the number and the unit are the two useful pieces of information.
WORD	`[A-Za-z]+(?:[-'][A-Za-z0-9]+)*`	Hyphenated terms (post-op, beta-blocker) and possessives (patient's) stay whole: these are words for search.
PUNCT	`[^\sA-Za-z0-9]`	Any other single symbol is its own token, so that no character is ever lost.

2. Maximal-munch tokenisation compared with a whitespace split

Maximal munch means that at every position all sub-patterns compete and the longest match wins. I verified that my ordered alternation gives the same tokens as true longest-match on 200

records (**0 differences**). Where the order of alternatives is wrong the two behaviours differ, as the last row shows.

Table 13: Whitespace split versus maximal munch

Input	Whitespace split	Maximal munch
BP 140/90mmHg, pulse 88/min.	BP | 140/90mmHg, | pulse | 88/min.	BP | 140/90 | mmHg | , | pulse | 88 | / | min | .
Pt on b.i.d. 2.5mg dosing; post-op Hb:10.2g/dL	Pt | on | b.i.d. | 2.5mg | dosing; | post-op | Hb:10.2g/dL	Pt | on | b.i.d. | 2.5 | mg | dosing | ; | post-op | Hb | : | 10.2 | g | / | dL
Seen on 15/12/2015: patient's k/c/o DM, HTN.	Seen | on | 15/12/2015: | patient's | k/c/o | DM, | HTN.	Seen | on | 15/12/2015 | : | patient's | k | / | c | / | o | DM | , | HTN | .
12/03/2024 (NUM listed before DATE)	12/03 | / | 2024 ← ordered alternation, wrong order	12/03/2024 ← longest match

The whitespace split leaves punctuation stuck to words (“140/90mmHg,”, “HTN.”) and fuses numbers with units (“Hb:10.2g/dL”), so a dictionary lookup of “HTN” or “mmHg” would fail. Maximal munch also shows the cost of its choices: “1-0-1” is cut into five tokens and “k/c/o” into five, because no sub-pattern covers them; the abbreviation table below handles “k/c/o” before tokenisation.

3. Substitution rules for normalisation

The normaliser is a sequence of regular-expression substitutions applied in this order: (1) dates to ISO format, (2) units spaced and standardised, (3) phone numbers to bare digits, (4) abbreviation expansion with 44 clinical abbreviations (more than the thirty required), and (5) case folding. The order matters: abbreviations must be expanded **before** case folding because OD and od are not the same in every context, and dates must be rewritten before the number rules can damage them.

Table 14: The clinical abbreviation table used by rule (4)

Abbr.	Expansion	Abbr.	Expansion	Abbr.	Expansion
bd	twice daily	icu	intensive care unit	pt	patient
bp	blood pressure	im	intramuscular	qid	four times daily
c/o	complains of	inj	injection	rr	respiratory rate
cad	coronary artery disease	iv	intravenous	rta	road traffic accident
cap	capsule	k/c/o	known case of	rx	treatment
copd	chronic obstructive pulmonary disease	mi	myocardial infarction	s/p	status post
dm	diabetes mellitus	n/v	nausea and vomiting	sob	shortness of breath
dx	diagnosis	npo	nil by mouth	sos	if needed
fx	fracture	o/e	on examination	sx	symptoms
h/o	history of	od	once daily	tab	tablet
hb	haemoglobin	ot	operation theatre	tds	thrice daily
hr	heart rate	po	by mouth	tx	treatment
hs	at bedtime	post-op	postoperative	uti	urinary tract infection
htn	hypertension	pre-op	preoperative	wbc	white cell count
hx	history	prn	as needed		

Table 15: Complete before-and-after text of one discharge record (R0001)

Before

```
UHID 235848. 88 yrs abnormal patient admitted on 17 Oct 2022 to Ward 4B, Bed 18. Presented
with
    observation and readmit for 8 days. Underwent transfer; bladder was resolved without vein.
grade 2
    tumour. Diagnosis superficial infusion ( I21.4 ). Underwent serums; appoinntment was
recovered
    without scar. creatinine 1.4, WBC 17465 /cumm, BP 165/75 mmHg, temp 99.3 F. Patient
amputated well
    and was immobilised on 17/01/2016. ref 204518. Advised Amlodipine 50 mg q8h and
Azithromycin 1000 mg
     1-1-1. scale 8/10. Review ocular plaque after scan. Contact +91 71394 04084.
```

After

```
uhid 235848. 88 yrs abnormal patient admitted on 2022-10-17 to ward 4b, bed 18. presented with
    observation and readmit for 8 days. underwent transfer; bladder was resolved without vein.
grade 2
    tumour. diagnosis superficial infusion ( i21.4 ). underwent serums; appoinntment was
recovered
    without scar. creatinine 1.4, white cell count 17465 /cumm, blood pressure 165/75 mmhg,
```

```
temp 99.3 f.
     patient amputated well and was immobilised on 2016-01-17. ref 204518. advised amlodipine
50 mg q8h
    and azithromycin 1000 mg 1-1-1. scale 8/10. review ocular plaque after scan. contact
7139404084.
```

Note what changed: “17 Oct 2022” became 2022-10-17, “Ward 4B, Bed 18” became lower case, every letter was folded, and abbreviations such as BP, Hb or OD would have been expanded if present.

4. Irreversible normalisation decisions

Table 16: Decisions that destroy information

Decision	Information destroyed	Acceptable when	Not acceptable when
Case folding	MS (multiple sclerosis) and ms (millisecond) become one string; drug names lose Tall-Man lettering (hydrOXYzine)	Searching for symptoms and procedures	Medication safety checks where look-alike names are distinguished by capitals
Abbreviation expansion	OD becomes “once daily” even in “vision OD” (71 of 134 cases in my corpus are the right eye)	The abbreviation is unambiguous in its field (dosage field)	Whenever the context decides the meaning
Date canonicalisation	The original format (and with it a possible day/month swap in dd/mm vs mm/dd) is lost	All sources use dd/mm/yyyy	Records merged from sources with mm/dd/yyyy
Phone number cleaning	Separators and the +91 prefix are dropped	The number is used only for matching	The number must be dialled or printed as written
Unit spacing	“500mg” and “500 mg” become identical	Always	–

5. A requirement that no regular expression can express

The hospital wants to check that every opening bracket in a nested clinical plan has a matching closing bracket, for example `Plan: (admit (if BP < 90 (check lactate)))`. The language of correctly balanced brackets needs to **count** the open brackets without limit. A regular expression (and a finite automaton) has only a fixed, finite memory, so after reading more opening brackets than it has states it must forget how many are open. Task 4 turns this argument into a proof with the pumping lemma, and a measurement there shows the number of states growing with the nesting depth. In practice the hospital would use a pushdown automaton (a stack), which is outside this unit.

6. The tokenizer and normaliser as code

```
TOKEN_PARTS = [
 ('DATE',   r'\d{1,2}[/.-]\d{1,2}[/.-]\d{2,4}'),
 ('ABBR',   r'(?:[A-Za-z]{1,3}\.){2,}'),                    # b.i.d.  o.d.  i.v.
 ('NUM',    r'\d+(?:[.,]\d+)?(?:/\d+(?:[.,]\d+)?)?'),        # 38.5   140/90   1,200
 ('UNIT',   r'(?:mg|mcg|g|ml|IU|mmHg|mmol/L|mg/dL|g/dL|/cumm|%)(?![A-Za-z])'),
 ('WORD',   r"[A-Za-z]+(?:[-'][A-Za-z0-9]+)*"),              # post-op, patient's, beta-blocker
 ('PUNCT',  r'[^\sA-Za-z0-9]'),
]

TOKEN_RE = re.compile('|'.join('(?P<%s>%s)' % p for p in TOKEN_PARTS))

PART_RE = [(n, re.compile(p)) for n, p in TOKEN_PARTS]

def tokenize(text):
    """single-regular-expression tokenizer (ordered alternation)."""
    return [(m.lastgroup, m.group(0), m.start()) for m in TOKEN_RE.finditer(text)]

def maximal_munch(text):
    """true longest-match tokenizer: at every position all alternatives compete, the longest
wins."""
    i, out = 0, []
    while i < len(text):
        if text[i].isspace(): i += 1; continue
        best = None
        for n, rx in PART_RE:
            m = rx.match(text, i)
            if m and (best is None or m.end() > best[2]): best = (n, m.group(0), m.end())
        out.append((best[0], best[1], i)); i = best[2]
    return out

def norm_dates(t):
    t = re.sub(r'\b(\d{1,2}) (%s) ((?:19|20)\d{2})\b' % MONTHS, lambda m: '%s-%s-%02d' %
```

```
(m.group(3),
        MONTH_NUM[m.group(2)], int(m.group(1))), t)
    t = re.sub(r'\b(\d{2})[/-](\d{2})[/-]((?:19|20)\d{2})\b', r'\3-\2-\1', t)
    return t

def norm_units(t):
    t = re.sub(r'(\d)(mg|mcg|g|ml|IU)\b', r'\1 \2', t)
    t = re.sub(r'(\d)\s?°\s?C\b', r'\1 C', t); t = re.sub(r'(\d)\s?°\s?F\b', r'\1 F', t)
    t = re.sub(r'\bg/dl\b', 'g/dL', t, flags=re.I)
    return t

def norm_phones(t):
    return re.sub(r'(?<![\d-])(?:\+91 )?([6-9]\d{4})[- ]?(\d{5})(?!\d)', r'\1\2', t)

def expand_abbrev(t):
    keys = sorted(ABBREV, key=len, reverse=True)
    rx = re.compile(r'(?<![\w/-])(%s)(?![\w/-])' % '|'.join(re.escape(k) for k in keys), re.I)
    return rx.sub(lambda m: ABBREV[m.group(1).lower()], t)

def normalise(t):
    t = norm_dates(t); t = norm_units(t); t = norm_phones(t)
    t = expand_abbrev(t)
    return t.lower()
```

TASK 4 – Finite State Automata: Design, Non-determinism and Equivalence — 5 Marks

Guiding Enquiry

How do automata recognise exactly the same languages as regular expressions, and what does removing non-determinism cost?

Investigate and interpret:

- Define a finite automaton formally as the five-tuple (Q, Σ, δ, q_0, F), then design a deterministic automaton for at least four validators from your pattern inventory, giving both the transition diagram and the transition table for each.
- Design a non-deterministic automaton with ε-transitions for one entity whose format has optional parts, and trace its behaviour on one accepted and one rejected string, showing the set of states live at every step.
- Convert that NFA to a DFA by subset construction, showing the complete construction table, then minimise the DFA and report the state counts before and after.
- Apply Thompson's construction to turn one Task 2 regular expression into an automaton, and derive a regular expression back from one of your automata, demonstrating the equivalence in both directions.
- State the pumping lemma for regular languages and use it to prove that one specific clinical pattern is not regular — for example a note requiring equal numbers of opening and closing brackets — and relate the proof to your finding in Task 3.

Evidence Expected

Correct automata with full subset-construction and minimisation tables, a demonstrated two-way equivalence with regular expressions, and a valid non-regularity proof.

Ans:

Finite State Automata: Design, Non-determinism and Equivalence

1. Formal definition and five deterministic validators

A finite automaton is a five-tuple **(Q, Σ, δ, q_0, F)**: a finite set of states Q, an input alphabet Σ, a transition function δ : Q × Σ → Q (for a DFA; Q × (Σ ∪ {ε}) → 2^Q for an NFA), a start state $q_0 \in Q$ and a set of accepting states $F \subseteq Q$. To keep the tables readable the alphabet uses classes: <d> is any digit, <U> any upper-case letter, and other symbols are literal. A missing entry means the automaton moves to an implicit dead state and rejects.

I designed five DFAs for the validators PID, BP, DATE (dd/mm/yyyy shape), ICD-10 and WARD-BED. Each has at least six states. For each one I give the transition diagram, the transition table, and tests (including strings that must be rejected). The automata and the regular expressions agree on every match found in the corpus: BP **452/452**, ICD **427/427**, PID **228/228**.

DFA PID: Patient identifier PID-dddddd (11 states, accepting {q10})

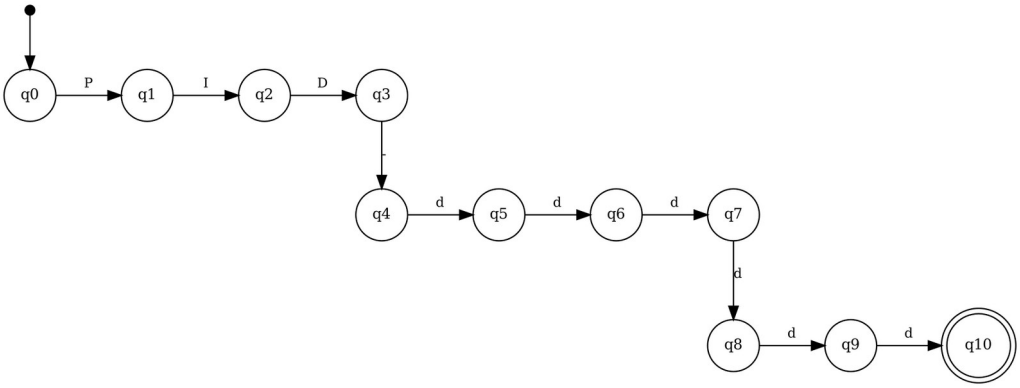


Figure 3: Transition diagram of the PID validator

Table 17: Transition table of the PID validator ("-" = dead state; d = digit, U = upper-case letter)

State	-	d	D	I	P
q0	-	-	-	-	q1
q1	-	-	-	q2	-
q2	-	-	q3	-	-
q3	q4	-	-	-	-
q4	-	q5	-	-	-
q5	-	q6	-	-	-
q6	-	q7	-	-	-
q7	-	q8	-	-	-
q8	-	q9	-	-	-
q9	-	q10	-	-	-
q10	-	-	-	-	-

Tests: `PID-305253` → **accept**; `PID-30525` → reject; `PID-3052533` → reject; `PIB-305253` → reject; `PID305253` → reject

DFA BP: Blood pressure ddd?/ddd? (8 states, accepting {q6, q7})

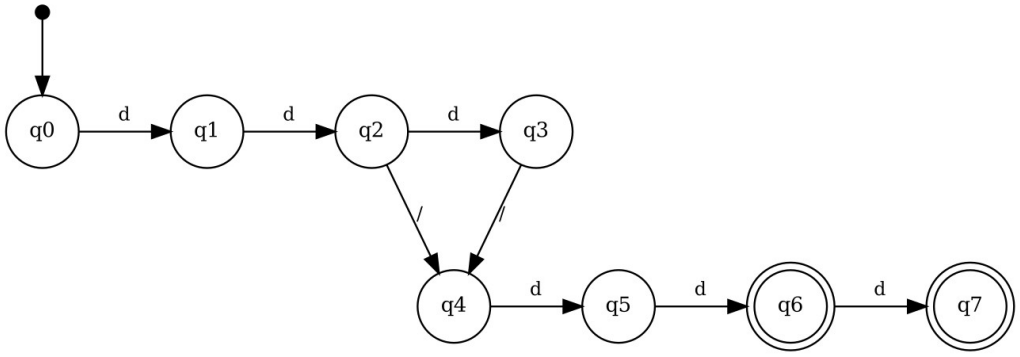


Figure 4: Transition diagram of the BP validator

Table 18: Transition table of the BP validator ("-" = dead state; d = digit, U = upper-case letter)

State	/	d
q0	-	q1
q1	-	q2
q2	q4	q3
q3	q4	-
q4	-	q5
q5	-	q6
q6	-	q7
q7	-	-

Tests: `140/90` → **accept**; `95/55` → **accept**; `1400/90` → reject; `14/9` → reject; `140/` → reject

DFA DATE: Date shape dd/mm/yyyy (11 states, accepting {q10})

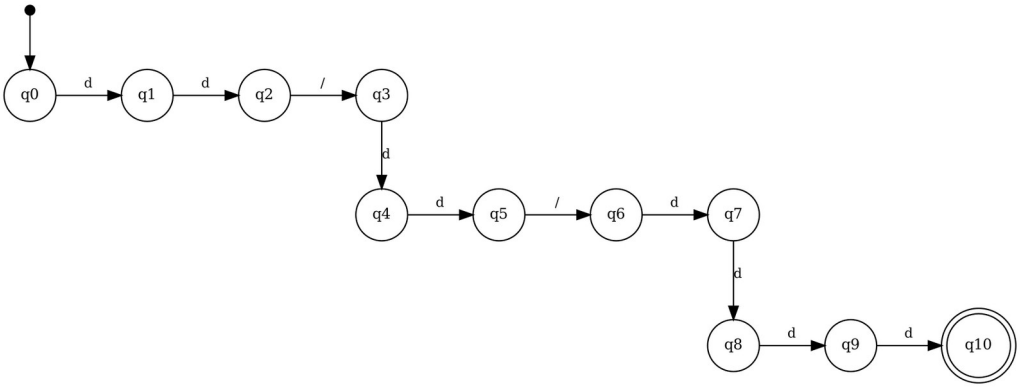


Figure 5: Transition diagram of the DATE validator

Table 19: Transition table of the DATE validator ("-" = dead state; d = digit, U = upper-case letter)

State	/	d
q0	-	q1
q1	-	q2
q2	q3	-
q3	-	q4
q4	-	q5
q5	q6	-
q6	-	q7
q7	-	q8
q8	-	q9
q9	-	q10
q10	-	-

Tests: `15/12/2015` → **accept**; `1/12/2015` → reject; `15-12-2015` → reject; `15/12/15` → reject

DFA ICD: ICD-10 code (9 states, accepting {q3, q5, q6, q7, q8})

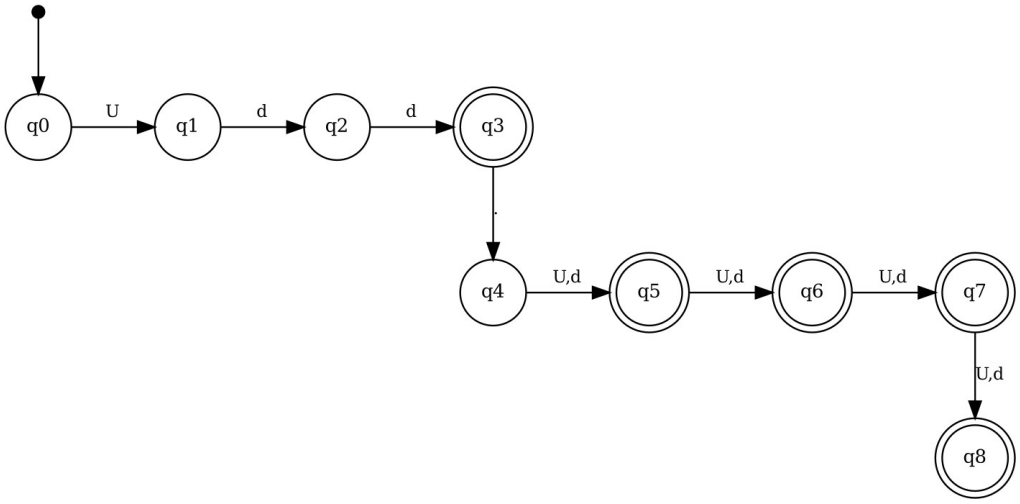


Figure 6: Transition diagram of the ICD validator

Table 20: Transition table of the ICD validator ("-" = dead state; d = digit, U = upper-case letter)

State	.	U	d
q0	-	q1	-
q1	-	-	q2
q2	-	-	q3
q3	q4	-	-
q4	-	q5	q5
q5	-	q6	q6
q6	-	q7	q7
q7	-	q8	q8
q8	-	-	-

Tests: `S72.001A` → **accept**; `I10` → **accept**; `I1` → reject; `K35.80XXX` → reject; `E11.9` → **accept**

DFA WARD: Ward and bed Wd[A-Z]?/Bdd? (8 states, accepting {q6, q7})

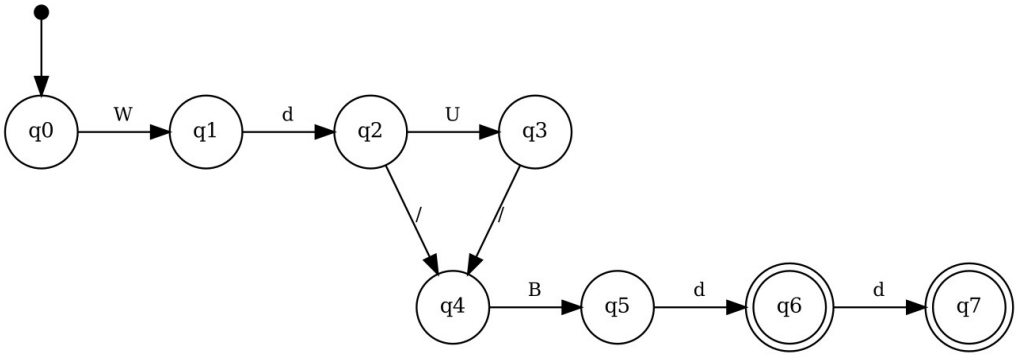


Figure 7: Transition diagram of the WARD validator

Table 21: Transition table of the WARD validator ("-" = dead state; d = digit, U = upper-case letter)

State	/	U	d	B	W
q0	-	-	-	-	q1
q1	-	-	q2	-	-
q2	q4	q3	-	-	-
q3	q4	-	-	-	-
q4	-	-	-	q5	-
q5	-	-	q6	-	-
q6	-	-	q7	-	-
q7	-	-	-	-	-

Tests: `W4B/B12` → **accept**; `W4/B3` → **accept**; `W4B/12` → reject

2. A non-deterministic automaton with ε-transitions: the contact number

Indian mobile numbers are written in several ways: with or without the prefix "+91 ", with a space, a hyphen or nothing between the two five-digit halves. These optional parts are naturally expressed with ε-moves: from the start state the machine may jump over the whole prefix (0 → 4 on ε), and after the prefix it may skip the space (3 → 4 on ε) or the separator in the middle (9 → 10 on ε). The alphabet is {+, 9, 1, d, s}, where d is any digit other than 9 or 1 and s is a space or hyphen; 9 and 1 are kept separate because the prefix uses them as literals.

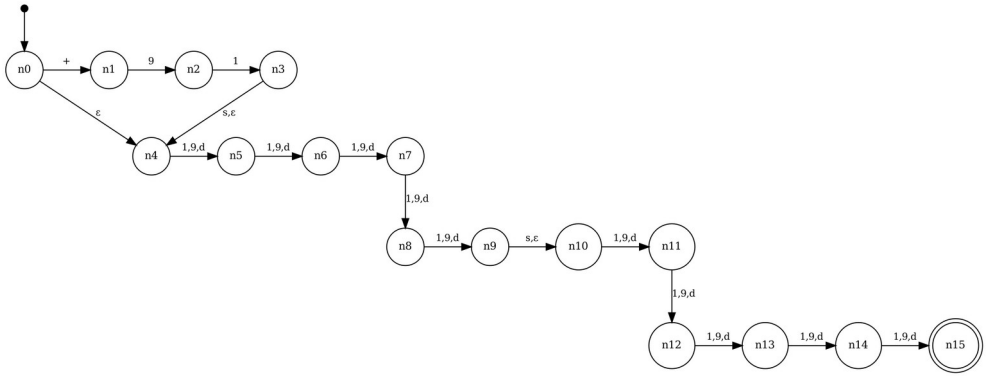


Figure 8: ε-NFA for contact numbers (16 states; accepting state n15)

Table 22: Trace of the NFA on the accepted string "98765-43210" (live states after every symbol)

Step	Symbol read	Live states
0	(start; ε-closure)	{0, 4}
1	'9'	{5}
2	'8'	{6}
3	'7'	{7}
4	'6'	{8}
5	'5'	{9, 10}
6	'-'	{10}
7	'4'	{11}
8	'3'	{12}
9	'2'	{13}
10	'1'	{14}
11	'0'	{15}

The final live set contains the accepting state 15, so the string is **accepted**.

Table 23: Trace of the NFA on the rejected string "+91 98765 4321" (one digit missing)

Step	Symbol read	Live states
0	(start; ε-closure)	{0, 4}
1	'+'	{1}
2	'9'	{2}
3	'1'	{3, 4}
4	' '	{4}
5	'9'	{5}
6	'8'	{6}
7	'7'	{7}
8	'6'	{8}
9	'5'	{9, 10}
10	' '	{10}
11	'4'	{11}
12	'3'	{12}
13	'2'	{13}
14	'1'	{14}

After the last symbol the live set {14} does not contain the accepting state 15, so the string is **rejected**: the machine was one digit short.

3. Subset construction and minimisation

I converted the NFA to a DFA with the subset construction. Each DFA state is the ε-closure of a set of NFA states; the start state is the closure of {0} = {0, 4}. The construction produced **16 DFA states from 16 NFA states**: the NFA is almost deterministic once its ε-moves are folded in, so there is no exponential blow-up.

Table 24: Complete subset-construction table (A is the start state, P is accepting)

DFA state	NFA states	+	9	1	d	space/-
A	{0, 4}	B	C	C	C	-
B	{1}	-	D	-	-	-
C	{5}	-	E	E	E	-
D	{2}	-	-	F	-	-
E	{6}	-	G	G	G	-
F	{3, 4}	-	C	C	C	H
G	{7}	-	I	I	I	-
H	{4}	-	C	C	C	-
I	{8}	-	J	J	J	-
J	{9, 10}	-	K	K	K	L
K	{11}	-	M	M	M	-
L	{10}	-	K	K	K	-
M	{12}	-	N	N	N	-
N	{13}	-	O	O	O	-
O	{14}	-	P	P	P	-
P	{15}	-	-	-	-	-

I then minimised the DFA by partition refinement (Moore's algorithm): start from {accepting} and {non-accepting}, and split blocks whose members go to different blocks on some symbol. The number of blocks per round was 2, 3, 4, 5, 6, 7, 9, 10, 11, 12, 13, 15, 16, 17. The result has **16 states: the DFA was already minimal** (16 before, 16 after), because each digit position must be remembered. To show minimisation actually shrinking a machine, I applied the same pipeline to a regular expression whose automaton has redundant tails, the set of dosage patterns `1-0-1|1-1-1|0-0-1|1-1-0|0-1-0`: Thompson's construction gives a 58-state ε-NFA, the subset construction a 18-state DFA, and minimisation merges states with identical futures (for example the states reached after "1-0-" and "0-0-" both expect only "1"), leaving **12 states** (a reduction of 6 states).

4. Equivalence with regular expressions in both directions

Regular expression → automaton (Thompson). I implemented Thompson's construction (each operator becomes a small ε-NFA fragment with one start and one accepting state). For the lab-value regex `d+(.d)?` (one or more digits, then optionally a point and a digit) it builds the 10-state ε-NFA of Figure 9. I checked it against Python's `re` on 3000 random strings over the alphabet {0–9, .}: **3000/3000 agreements**.

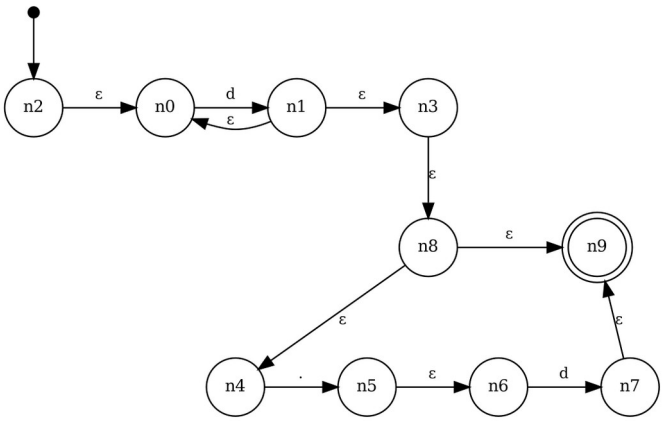


Figure 9: Thompson ε-NFA for the pattern d+(.d)?

Automaton → regular expression (state elimination). I removed the states of the BP DFA one by one, replacing paths through the eliminated state by regular-expression labels. The result is

```
((((dd/|ddd/)d)d)|((((dd/|ddd/)d)d)d))
```

which is equivalent to d d d? / d d d? written out. Converting it to a Python regex and comparing with the DFA on 3000 random strings over {0–9, /} gave **3000/3000 agreements**. So the regular expression, the NFA and the DFA all denote the same language.

5. The pumping lemma and a non-regular clinical pattern

Pumping lemma. If L is a regular language, there is a number p ≥ 1 (the pumping length) such that every string w ∈ L with |w| ≥ p can be written w = xyz with (1) |xy| ≤ p, (2) |y| ≥ 1, and (3) xy^i z ∈ L for every i ≥ 0.

Claim. L = { (^n)^n : n ≥ 0 } (n opening brackets followed by n closing brackets, the core of balanced nested notes) is not regular.

Proof. Suppose L is regular with pumping length p. Take w = (^p)^p ∈ L, |w| = 2p ≥ p. By (1), xy lies inside the first p characters, which are all “(”, so y = (^k with k ≥ 1 by (2). By (3) with i = 0 the string xz = (^(p−k))^p must be in L, but it has p − k opening and p closing brackets, and p − k ≠ p. So xz ∉ L, a contradiction. Hence L is not regular. ∎

Link to Task 3 and to the data. The proof says any finite automaton must have more than p states to count p brackets. I confirmed this by building the minimal DFA that accepts balanced strings of depth at most n: its size grows as 2n (Figure 10). No fixed automaton, and therefore no regular expression, can validate arbitrary nesting; this is exactly why Task 3 concluded that bracket matching lies outside a purely regular layer.

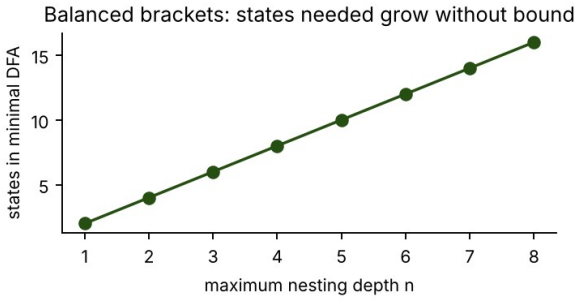


Figure 10: States of the minimal DFA for balanced brackets of depth at most n

6. The automata as code

```
def cls(ch):
    if ch.isdigit(): return '<d>'
    if ch.isupper(): return '<U>'
    if ch.islower(): return '<L>'
    return ch

class DFA:
    """(Q, Sigma, delta, q0, F).  Symbols are literal characters or the classes <d> (digit), <U>
        (upper), <L> (lower)."""
```

```
    def __init__(self, name, Q, delta, q0, F):
        self.name, self.Q, self.delta, self.q0, self.F = name, list(Q), delta, q0, set(F)
    def step(self, q, ch):
        for sym in (ch, cls(ch)):
            if (q, sym) in self.delta: return self.delta[(q, sym)]
        return None
    def accepts(self, s):
        q = self.q0
        for ch in s:
            q = self.step(q, ch)
            if q is None: return False
        return q in self.F
    def symbols(self):
        return sorted({a for (_, a) in self.delta})
    def table(self):
        syms = self.symbols()
        return syms, [[q] + [self.delta.get((q, a), '-') for a in syms] for q in self.Q]

def make_validators():
    d_, U_ = '<d>', '<U>'
    V = {}
    # 1. patient identifier  PID-dddddd
    d = {('q0', 'P'): 'q1', ('q1', 'I'): 'q2', ('q2', 'D'): 'q3', ('q3', '-'): 'q4'}
    for i in range(6): d[('q%d' % (4 + i), d_)] = 'q%d' % (5 + i)
    V['PID'] = DFA('PID', ['q%d' % i for i in range(11)], d, 'q0', {'q10'})
    # 2. blood pressure  ddd?/ddd?   (two or three digits either side of the slash)
    d = {('q0', d_): 'q1', ('q1', d_): 'q2', ('q2', d_): 'q3', ('q2', '/'): 'q4', ('q3', '/'): 'q4',
         ('q4', d_): 'q5', ('q5', d_): 'q6', ('q6', d_): 'q7'}
    V['BP'] = DFA('BP', ['q%d' % i for i in range(8)], d, 'q0', {'q6', 'q7'})
    # 3. date shape dd/mm/yyyy
    d = {}
    for i, a in enumerate([d_, d_, '/', d_, d_, '/', d_, d_, d_, d_]): d[('q%d' % i, a)] = 'q%d' %
(i +
        1)
    V['DATE'] = DFA('DATE', ['q%d' % i for i in range(11)], d, 'q0', {'q10'})
    # 4. ICD-10  U dd [. (d|U){1,4}]
    d = {('q0', U_): 'q1', ('q1', d_): 'q2', ('q2', d_): 'q3', ('q3', '.'): 'q4'}
    for i in range(4):
        for a in (d_, U_): d[('q%d' % (4 + i), a)] = 'q%d' % (5 + i)
    V['ICD'] = DFA('ICD', ['q%d' % i for i in range(9)], d, 'q0', {'q3', 'q5', 'q6', 'q7', 'q8'})
    # 5. ward-bed  W d [A-Z]? / B d d?
    d = {('q0', 'W'): 'q1', ('q1', d_): 'q2', ('q2', U_): 'q3', ('q2', '/'): 'q4', ('q3', '/'):
'q4',
         ('q4', 'B'): 'q5', ('q5', d_): 'q6', ('q6', d_): 'q7'}
    V['WARD'] = DFA('WARD', ['q%d' % i for i in range(8)], d, 'q0', {'q6', 'q7'})
    return V

class NFA:
    """(Q, Sigma, delta, q0, F) with epsilon moves ('')."""
    def __init__(self, Q, delta, q0, F):
        self.Q, self.delta, self.q0, self.F = list(Q), delta, q0, set(F)
    def eclose(self, S):
        stack, seen = list(S), set(S)
        while stack:
            q = stack.pop()
            for r in self.delta.get((q, ''), ()):
                if r not in seen: seen.add(r); stack.append(r)
        return frozenset(seen)
    def move(self, S, a):
        return self.eclose({r for q in S for r in self.delta.get((q, a), ())})
    def trace(self, s, sym=lambda c: c):
        S = self.eclose({self.q0}); rows = [('', sorted(S))]
        for ch in s:
            S = self.move(S, sym(ch)); rows.append((ch, sorted(S)))
        return rows, bool(S & self.F)

def phone_nfa():
    """[+91 ]? ddddd [ -]? ddddd  with epsilon moves for the optional parts.  Symbols: + 9 1 d s
        (separator)."""
    D = {}
    def add(q, a, r): D.setdefault((q, a), set()).add(r)
    add(0, '+', 1); add(1, '9', 2); add(2, '1', 3); add(3, 's', 4); add(3, '', 4); add(0, '', 4)
    for i in range(5):
        for a in ('9', '1', 'd'): add(4 + i, a, 5 + i)
    add(9, 's', 10); add(9, '', 10)
    for i in range(5):
        for a in ('9', '1', 'd'): add(10 + i, a, 11 + i)
    return NFA(range(16), D, 0, {15})

def subset_construction(nfa, alphabet):
    start = nfa.eclose({nfa.q0}); names = {start: 'A'}; order = [start]; rows = []; delta = {}
    i = 0
    while i < len(order):
```

```
        S = order[i]; i += 1; row = []
        for a in alphabet:
            T = nfa.move(S, a)
            if not T: row.append('-'); continue
            if T not in names: names[T] = chr(ord('A') + len(names)) if len(names) < 26 else 'S%d' %
                len(names); order.append(T)
            row.append(names[T]); delta[(names[S], a)] = names[T]
        rows.append((names[S], sorted(S), row))
    F = {names[S] for S in order if S & nfa.F}
    return rows, delta, set(names.values()), F

def minimise(Q, alphabet, delta, q0, F):
    """Moore partition refinement on the completed DFA.  Returns (n_min_states, rounds)."""
    dead = '#'; Qc = list(Q) + [dead]
    d = lambda q, a: delta.get((q, a), dead) if q != dead else dead
    part = {q: (q in F) for q in Qc}; rounds = []
    while True:
        blocks = {}
        for q in Qc: blocks.setdefault(part[q], []).append(q)
        rounds.append([sorted(b) for b in blocks.values()])
        sig = {q: (part[q],) + tuple(part[d(q, a)] for a in alphabet) for q in Qc}
        ids = {}; new = {}
        for q in Qc: new[q] = ids.setdefault(sig[q], len(ids))
        if len(set(new.values())) == len(set(part.values())): break
        part = new
    # drop blocks that are only the dead state / unreachable (reachability from q0)
    reach, st = {q0}, [q0]
    while st:
        q = st.pop()
        for a in alphabet:
            r = d(q, a)
            if r not in reach: reach.add(r); st.append(r)
    blocks_final = {part[q] for q in reach if q != dead}
    return len(blocks_final), rounds

class Thompson:
    """mini regex: literals, d (digit class), ( ) | * + ?  — builds an epsilon-NFA by Thompson's
        rules."""
    def __init__(self, rx):
        self.rx, self.i, self.n, self.delta = rx, 0, 0, {}
        f = self.expr(); self.start, self.accept = f.start, f.end
    def new(self): self.n += 1; return self.n - 1
    def add(self, q, a, r): self.delta.setdefault((q, a), set()).add(r)
    def peek(self): return self.rx[self.i] if self.i < len(self.rx) else None
    def expr(self):
        f = self.term()
        while self.peek() == '|':
            self.i += 1; g = self.term(); s, e = self.new(), self.new()
            self.add(s, '', f.start); self.add(s, '', g.start); self.add(f.end, '', e);
self.add(g.end,
                '', e); f = Frag(s, e)
        return f
    def term(self):
        f = None
        while self.peek() not in (None, '|', ')'):
            g = self.factor()
            if f is None: f = g
            else: self.add(f.end, '', g.start); f = Frag(f.start, g.end)
        return f if f else self.lit('')
    def factor(self):
        if self.peek() == '(':
            self.i += 1; f = self.expr(); self.i += 1
        else:
            c = self.peek(); self.i += 1; f = self.lit(c)
        while self.peek() in ('*', '+', '?'):
            op = self.peek(); self.i += 1; s, e = self.new(), self.new()
            self.add(s, '', f.start); self.add(f.end, '', e)
            if op in ('*', '?'): self.add(s, '', e)
            if op in ('*', '+'): self.add(f.end, '', f.start)
            f = Frag(s, e)
        return f
    def lit(self, c):
        s, e = self.new(), self.new(); self.add(s, c, e); return Frag(s, e)
    def nfa(self): return NFA(range(self.n), self.delta, self.start, {self.accept})

def dfa_to_regex(dfa):
    states = list(dfa.Q); S, A = 'S*', 'A*'
    R = {}
    def u(a, b):
        if a is None: return b
        if b is None: return a
        return a if a == b else '(%s|%s)' % (a, b)
    for (q, a), r in dfa.delta.items():
```

```
        R[(q, r)] = u(R.get((q, r)), a)
    R[(S, dfa.q0)] = ''
    for f in dfa.F: R[(f, A)] = ''
    def cat(*xs):
        return ''.join(x if (len(x) <= 1 or x.startswith('(') and x.endswith(')') and x.count('(')
== 1
            or '|' not in x) else '(%s)' % x for x in xs)
    for q in states:
        loop = R.pop((q, q), None)
        ins = [(p, R[(p, q)]) for (p, r) in list(R) if r == q and p != q]
        outs = [(r, R[(q, r)]) for (p, r) in list(R) if p == q and r != q]
        for p, _ in ins: R.pop((p, q))
        for r, _ in outs: R.pop((q, r))
        lp = ('(%s)*' % loop if loop else '')
        for p, a in ins:
            for r, b in outs:
                R[(p, r)] = u(R.get((p, r)), cat(a, lp, b))
    return R.get((S, A))
```

TASK 5 – Inflectional Morphology **3 Marks**

Guiding Enquiry

Which inflectional processes appear in clinical English, and how are surface forms systematically related to their stems?

Investigate and interpret:

- Define morpheme, stem, affix, inflection and paradigm, and separate inflection from derivation using the standard criteria of category preservation, productivity and position relative to other affixes.
- List every inflectional paradigm present in your corpus — noun number, verb tense and aspect, subject agreement, adjective comparison — and give the full paradigm for at least five clinical lexemes.
- State the orthographic rules governing English inflection — e-deletion, consonant doubling, y-to-i, the sibilant -es rule — precisely enough to be implemented, and give for each one a counter-example on which it over-generates.
- Identify the irregular forms in your corpus and explain why each must be listed in the lexicon rather than derived by rule.
- Measure the effect of inflection: report the type-to-lemma ratio of your corpus and the proportion of failures in your query set that inflection alone explains.

Evidence Expected

Precisely stated inflectional rules with paradigms, an irregular list, and a quantified measurement of the effect of inflection on retrieval.

Ans:

Inflectional Morphology

1. Definitions and how inflection differs from derivation

Table 25: Key terms

Term	Definition	Clinical example
Morpheme	The smallest meaning-bearing unit	fracture, -s, -ed, nephr-, -itis
Stem	The part an affix attaches to	fracture in fractured; nephr in nephritis
Affix	A bound morpheme added to a stem (prefix or suffix)	-ing, un-, hyper-
Inflection	Adds grammatical information (number, tense, degree) without changing the word's meaning or category	fracture → fractures, fractured, fracturing
Paradigm	The full set of inflected forms of one lexeme	bleed, bleeds, bled, bled, bleeding

Table 26: Inflection versus derivation: the three criteria

Criterion	Inflection	Derivation
Category preservation	Keeps the category: treat (V) → treated (V)	Often changes it: treat (V) → treatment (N) → treatable (A)
Productivity	Applies to (almost) every stem of the class: any noun takes plural	Limited and lexically selective: -itis attaches to organ roots, not to “bleed”
Position	Outermost: it is added **last** (nephrectomies = nephr + ectomy + **ies**)	Inside inflection: derivation is added first

2. Inflectional paradigms in my corpus

Clinical English uses four inflectional processes. I list the affix (or allomorph) entries stored in the lexicon; there are 22 in all.

Table 27: Inflectional paradigms and the 22 affix entries

Paradigm	Affixes / allomorphs	Count
Noun number	-s, -es, -ies, -ves, -a (bacteria), -ae (vertebrae), -i (bronchi), -ices (appendices), -es (diagnoses), -en (children), vowel change (teeth, feet), -ø (data)	12
Verb 3rd-person singular	-s, -es, -ies	3
Verb past and past participle	-ed, -d, -ied (regular); listed forms for irregulars	3
Verb progressive	-ing	1
Adjective comparison	-er, -est, and suppletive better/best, worse/worst	3

The full paradigms of seven lexemes (computed by the rule cascade, not typed in) are:

Table 28: Full paradigms of clinical lexemes (generated by the system)

Lexeme	Slots	Forms
fracture (N)	SG, PL	fracture, fractures
fracture (V)	BASE, 3SG, PAST, PP, PROG	fracture, fractures, fractured, fractured, fracturing
biopsy (N/V)	SG, PL, 3SG, PAST, PROG	biopsy, biopsies, biopsies, biopsied, biopsying
bleed (V)	BASE, 3SG, PAST, PP, PROG	bleed, bleeds, bled, bled, bleeding
admit (V)	BASE, 3SG, PAST, PP, PROG	admit, admits, admitted, admitted, admitting
mild (A)	BASE, COMP, SUP	mild, milder, mildest
diagnosis (N)	SG, PL	diagnosis, diagnoses

3. Orthographic rules and where each one over-generates

The spelling changes at a morpheme boundary (written ^) are captured by seven rules in the form a → b / c __ d. Each rule is precise enough to implement (they are the rules of Task 8). A rule stated too generally **over-generates**; the last column shows the wrong output of the unconstrained version on a counter-example, computed by running it.

Table 29: Inflectional spelling rules with counter-examples

Rule	Statement	Example	Counter-example for the general rule	Wrong output
E-deletion	e → ø / [^eoy] __ ^ V	fracture^ed → fractured	see^ing	seing (should be seeing)
Consonant doubling	C → CC / V C __ ^ [ei] (stressed final syllable only)	admit^ed → admitted	visit^ed	visitted (should be visited)
y → i	y → ie / C __ ^ s; y → i / C __ ^ (ed|er|est|ness|ly)	biopsy^s → biopsies	stay^ed (vowel before y)	staied (should be stayed)
Sibilant -es	ø → e / (s|x|z|ch|sh) ^ __ s	dress^s → dresses	stomach^s (ch = /k/)	stomaches (should be stomachs)
f → v	f(e) → ve / (kni|wi|li|lea|hal|cal) __ ^ s	knife^s → knives	belief^s	believs (should be beliefs)
k-insertion	ø → k / c __ ^ [ei] (panic, mimic)	panic^ed → panicked	arc^ed	arcked (should be arced)

4. Irregular forms

The corpus contains 570 tokens of irregular inflected forms. They are listed separately in the lexicon (12 nouns, 30 verbs, 3 adjectives) because **no rule derives them**: the vowel change in took, ran or teeth, the suppletion in worse (from bad/ill) and the Latin plurals in bronchi or bacteria are historical accidents. A rule that tried to cover them would also fire on regular words (a rule "-oo- → -ee-" would make "tooth → teeth" but "booth → beeth"), so the list is both shorter and safer.

Table 30: The most frequent irregular forms in the corpus

Surface form	Lemma	Feature	Tokens
bronchi	bronchus	PL	26
lost	lose	PAST	22
lost	lose	PP	20
ran	run	PAST	19
run	run	PP	18
taken	take	PP	17
took	take	PAST	16
spread	spread	PAST	16
saw	see	PAST	15
drunk	drink	PP	13
swollen	swell	PP	13
children	child	PL	12
cut	cut	PP	12
drank	drink	PAST	12

5. Measuring the effect of inflection

Type-to-lemma ratio. The corpus has 1,645 word types; the transducer analyses 1,101 of them and assigns them to 507 lemmas, so each lemma appears on average in **2.17 different surface forms**. A search on the base form alone therefore sees only about 46% of the vocabulary of a lexeme.

Retrieval failures. With no normalisation, my 40 queries miss 473 relevant (query, record) pairs. I then re-ran the queries with inflection-only analysis, and with full analysis, and counted which misses each recovers:

Table 31: What explains the misses of exact-match search

Cause of the miss	Pairs	Share
Inflection alone (recovered by inflection-only analysis)	404	85.4%
Derivation (needs conflation of derived words, e.g. operation ↔ operate)	48	10.1%
Spelling variants, typos, out-of-lexicon words	21	4.4%

Inflection alone explains 85.4% of the failures. This is the single biggest reason why a clinician searching for "fracture" does not find "fractures", and it is why the finite-state layer pays for itself even before derivation is considered.

TASK 6 – Derivational Morphology — 4 Marks

Guiding Enquiry

How does derivation build the specialised clinical vocabulary, and why is it harder to handle than inflection?

Investigate and interpret:

- Define derivation and list at least fifteen derivational affixes attested in your corpus — for example -itis, -ectomy, -ology, -osis, -aemia, -ation, -able, -ive, nephro-, hyper-, hypo-, re-, un- — stating the category change each one causes.
- Build the derivational family of at least six clinical roots, showing the order in which affixes attach and identifying at least two orders that are impossible.

- Explain semantic non-compositionality with clinical examples in which the meaning of the derived word cannot be predicted from its parts, and state what this implies for a search system that stems aggressively.
- Compare the productivity of your affixes empirically: count types and tokens for each in the corpus and classify each affix as productive or fossilised, with the evidence for the classification.
- Decide whether derivational relatives should be conflated for search, citing at least two cases where conflation helps the clinician and two where it produces a clinically misleading match.

Evidence Expected

A documented derivational analysis with an affix inventory, productivity measurements and a reasoned position on conflation.

Ans:

Derivational Morphology

1. Derivation and the affix inventory

Derivation builds a new lexeme from a stem by adding an affix; it usually changes the category or the meaning and is lexically selective (-itis attaches to organ roots, not to verbs). Clinical vocabulary is largely built this way. The lexicon holds 20 derivational affixes: 8 attach to bound combining roots (nephr-, hepat-, arthr-…), the rest to ordinary words. The "conflate" column is my search policy (Section 5).

Table 32: Derivational affix inventory, category change and productivity measurements

Affix	Meaning	Category change	Lexicon types	Corpus types	Corpus tokens	Hapax	Class	Conflate
-itis	inflammation of	root->N	17	17	98	0	productive	no
-ectomy	surgical removal of	root->N	7	7	53	0	productive	no
-(o)logy	study of (-o-logy)	root->N	8	8	54	0	productive	no
-osis	abnormal condition of	root->N	3	3	17	0	restricted	no
-(o)tomy	surgical cut into (-o-tomy)	root->N	5	5	39	0	productive	no
-algia	pain in	root->N	4	4	22	0	restricted	no
-pathy	disease of	root->N	6	6	39	0	productive	no
-aemia	condition of the blood	root->N	3	3	24	0	restricted	no
-ation	act/result of V	V->N	5	5	33	0	productive	yes
-ion	act/result of V	V->N	7	7	40	0	productive	yes
-ment	act/result of V	V->N	3	3	16	0	restricted	yes
-able	able to be V-ed	V->A	2	2	18	0	restricted	yes
-ive	tending to V	V->A	3	3	20	0	restricted	yes
-ness	state of being A	A->N	2	2	19	0	restricted	yes
-ly	in an A manner	A->Adv	3	3	10	0	restricted	yes
-ity	state of being A	A->N	1	1	5	0	fossilised	yes
un-	not	A->A	2	2	15	0	restricted	no
re-	again	V->V	2	2	13	0	restricted	yes
hyper-	above normal	N->N	3	3	16	0	restricted	no
hypo-	below normal	N->N	3	3	20	0	restricted	no

2. Derivational families and impossible affix orders

Affixes attach in a fixed order: **root → derivational suffix → inflectional suffix**. Prefixes attach outside the word they modify. The families below were generated by the system from the bound roots; the cardio- family loses its linking vowel before a vowel-initial suffix, and a root that ends in the stem of the next affix (tonsill-) is stored with its doubled consonant.

Table 33: Derivational families of six clinical roots

Root	Meaning	Derived words (affix order: root + affix)
nephr-	kidney	nephritis, nephrectomy, nephrology, nephrosis, nephropathy
hepat-	liver	hepatitis, hepatology
arthr-	joint	arthritis, arthrosis, arthralgia
cardi-	heart	cardiology, cardialgia, cardiopathy
gastr-	stomach	gastritis, gastrectomy, gastrotomy

Root	Meaning	Derived words (affix order: root + affix)
neur-	nerve	neuritis, neurology, neurosis, neuralgia, neuropathy

Table 34: Orders that are impossible

Attempted order	Why it is impossible
*nephr + itis + ectomy (“nephritisectomy”)	Both are category-forming suffixes on a bound root; -ectomy needs the root, not an already formed noun. The correct word is nephr + ectomy.
*treat + ment + able → treatmentable	The suffix -able selects a verb, but treat + ment is already a noun. The correct word is treat + able.
*admit + re + ion (“admitreion”)	A prefix must attach to the left edge: re + admit + ion = readmission.
*tension + hyper → tensionhyper	Prefixes are only added on the left: hyper + tension.
*nephr + itis + s + ology	Inflection (-s) is outermost; nothing can be derived after it.

3. Semantic non-compositionality

The meaning of a derived word often cannot be computed from its parts, which is why derivation is harder to handle than inflection (inflection changes only a grammatical value, so the meaning stays fixed).

Table 35: Meaning that is not predictable from the parts

Word	Prediction from the parts	Actual clinical meaning
administration	administer + -ation: the act of administering (giving a drug)	Also the hospital management office
operation	operate + -ion: the act of operating a device	A surgical procedure
observation	observe + -ation: the act of observing	A ward status: a patient kept under watch
discharge	dis + charge: to remove a load	Release from hospital; also fluid from a wound
resident	reside + -ent: someone who lives somewhere	A doctor in training
culture	cult + -ure	Growing micro-organisms from a specimen

Implication for search. An aggressive stemmer treats administration, administer and administrative as one word. A clinician searching for administration of a drug would also get notes about hospital administration. Therefore conflation must be selective, not automatic: a stemmer, which has no lexicon, cannot know which derived words are semantically transparent.

4. Productivity of the affixes

I measured productivity in two ways. **Type frequency** is the number of different bases an affix attaches to in my lexicon: an affix attached to five or more bases is productive, two to four is restricted, and one is fossilised. **Corpus evidence** is the number of distinct word types and tokens actually attested and the hapax count (types occurring once); the hapax ratio is the standard measure of whether new words keep being formed.

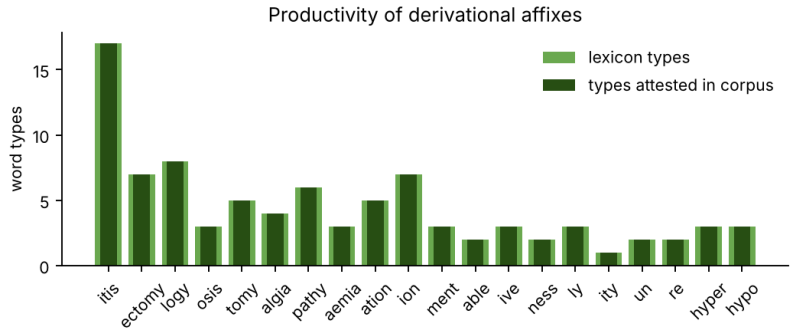


Figure 11: Lexicon types and corpus-attested types per derivational affix

Classification: productive: -itis, -ectomy, -logy, -tomy, -pathy, -ation, -ion; restricted: osis, algia, aemia, ment, able, ive, ness, ly, un, re, hyper, hypo; fossilised in this lexicon: ity. Almost every hapax count is zero, because my 520 records repeat a small vocabulary; so the type frequency is

the better evidence here. Note that -ly, -ness and un- are productive in general English; in this clinical lexicon they attach to only a few bases, so I call them restricted.

5. Should derivational relatives be conflated for search?

I decided **selectively**. Derivatives that are the same clinical concept with a different grammatical role are conflated (-ation, -ion, -ment, -able, -ive, -ness, -ly, -ity, re-). Derivatives that name a different disease, procedure or direction are kept apart (-itis, -ectomy, -(o)logy, -osis, -algia, -pathy, -aemia, hyper-, hypo-, un-).

Table 36: Where conflation helps and where it misleads

Case	Words	Effect on the clinician
Helps	treat, treated, treatment, treatable	A search for "treatment" also finds notes that say "treated" – the same plan.
Helps	operate, operation, operative, reoperate	All records about the surgery of one patient are found with one query.
Misleads	hypertension ↔ hypotension (same root tension)	Opposite conditions; conflating them gives a clinically dangerous false match. My transducer does **not** conflate them.
Misleads	nephritis ↔ nephrectomy (same root nephr-)	An inflammation and a surgical removal; conflation would flood a search for nephrectomy with nephritis notes.
Misleads (Porter)	dress, dressing; ventilate, ventilator	Porter gives both pairs one stem: a wound dressing is not a verb of clothing, and a ventilator (a machine) is not the act of ventilating.

TASK 7 – Finite-State Morphological Parsing 5 Marks

Guiding Enquiry

How can a transducer relate a surface form to its stem and features, and run in both directions on the same machine?

Investigate and interpret:

- Define a finite-state transducer as a two-tape automaton and explain the lexical and surface tapes with one fully worked clinical example.
- Build your lexicon as an FST organised into continuation classes, covering at least three parts of speech together with the inflectional and derivational affixes of Tasks 5 and 6.
- Hand-trace the parsing of at least five surface forms, showing the state sequence, the symbols consumed and emitted on each tape, and the final analysis — for example fractures → fracture +N +PL.
- Run the same transducer in the generation direction, producing surface forms from stem-plus-feature input, and show explicitly that parsing and generation use one machine and not two.
- Handle ambiguity: give at least two surface forms with more than one analysis, show the transducer returning all of them, and state how a downstream component would choose between them.

Evidence Expected

A working transducer specification with traced analyses in both directions and an explicit treatment of ambiguity.

Ans:

Finite-State Morphological Parsing

1. The transducer as a two-tape automaton

A **finite-state transducer** is a finite automaton whose arcs carry **pairs of symbols** a : b. Reading the machine along the first coordinate gives the **lexical tape** (stem + features); the second coordinate gives the **surface tape** (the spelling in the record). Because every arc is a pair, the same machine can be read in either direction: input surface and output lexical (parsing), or input lexical and output surface (generation).

Worked example. The path for fractures ↔ fracture +N +PL uses eleven arcs:

Table 37: One fully worked path: lexical tape against surface tape

Arc	1	2	3	4	5	6	7	8	9	10
Lexical tape	f	r	a	c	t	u	r	e	+N	+PL
Surface tape	f	r	a	c	t	u	r	e	ε	s

The stem letters are copied (f:f, r:r, …); the feature tags +N and +PL are single lexical symbols that print nothing (+N:ε) or the plural suffix (+PL:s).

2. The lexicon as an FST with continuation classes

The lexicon is organised into **continuation classes**: each stem belongs to a class, and the class says which affix classes may follow. The classes cover three parts of speech (noun, verb, adjective) plus bound roots, derivational words and closed-class items. The inflectional affixes of Task 5 and the derivational affixes of Task 6 are the arcs of the affix classes. The orthographic rules of Task 8 are composed with this lexicon; compiling the whole composition gives one machine with **1,595 lexical:surface pairs, 6,799 states and 6,798 arcs**.

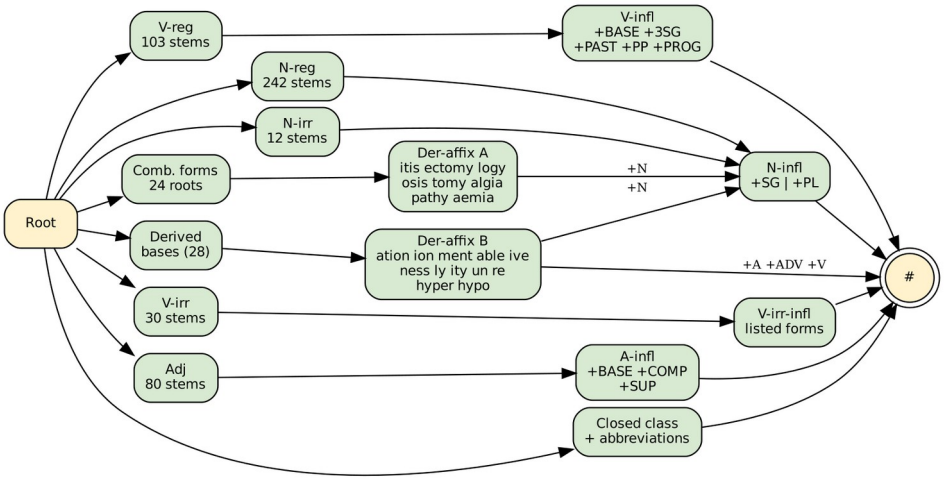


Figure 12: Continuation classes of the lexicon FST (# = end of word)

Table 38: Continuation classes

Class	Members (examples)	Continues with
N-reg (242)	fracture, kidney, wound, biopsy	N-infl: +SG (ε) | +PL (s, es, ies, ves)
N-irr (12)	diagnosis, bacterium, bronchus, child	listed +PL forms
V-reg (103)	treat, admit, discharge, infect	V-infl: +BASE, +3SG, +PAST, +PP, +PROG
V-irr (30)	bleed, take, see, undergo	listed +PAST and +PP forms
Adj (80)	mild, severe, renal, acute	A-infl: +BASE, +COMP, +SUP (gradable only)
Combining roots (24)	nephr-, hepat-, arthr-, tonsill-	Der-affix A (itis, ectomy, logy …) then N-infl
Derived bases (28)	examine, treat, weak, mild, tension	Der-affix B (ation, ment, ness, ly, hyper …) then N-infl or end
Closed class, abbreviations	and, with, was, bd, mg	end of word

3. Hand-traces of five (six) surface forms

Each trace follows the machine from state 0 through the arcs it takes while it **consumes** the surface string and **emits** the lexical tape (ε = nothing consumed or emitted). Only the first accepting path is shown when there are several.

Table 39: Parsing traces: state sequence, symbols consumed and emitted, final analysis

Surface	State sequence	Consumed (surface)	Emitted (lexical symbols)	Analysis
fractures	0 750 848 849 850 851 852 853 854 855 857 858	fractures	f r a c t u r e +N +PL	fracture+N+PL
biopsies	0 179 226 238 249 250 254 255 256 257 258 259	biopsies	b i o p s y +N +PL	biopsy+N+PL
bled	0 179 260 270 2519 2520 2521 2522 2523	bled	b l e e d +V +PAST	bleed+V+PAST
nephrectomies	0 1256 1266 6150 6151 6152 6153 … 6341 6342 6346 6347 6348 6349 6350	nephrectomies	n e p h r +ROOT +DER_ectomy +N +PL	nephr+ROOT+DER_ectomy+N+PL
admitted	0 1 2297 2298 2299 2320 2325 2326 2331 2332 2333	admitted	a d m i t +V +PAST	admit+V+PAST
examination	0 707 3015 3016 3017 3018 3019 … 5689 5690 5691 5692 5693 5694 5695	examination	e x a m i n e +V +DER_ation +N +SG	examine+V+DER_ation +N+SG

In every trace the stem letters are consumed and emitted unchanged (x:x). Irregular and rule-changed forms show different behaviour: in **bled**, the lexical letters “eed” are emitted while only “ed” is consumed (the arcs e:ε and e:ε drop the vowels that the surface does not contain); in **biopsies** the lexical y is emitted against the surface i (y:ε then ε:i ε:e), and the plural arc +PL:s completes the string.

4. The same machine in the generation direction

Generation feeds a lexical string into the **same arcs** from the other side. There is no second machine: the one object `FST` has two methods that differ only in which coordinate of each arc pair they match against the input.

Table 40: Generation: lexical input → surface output

Lexical input	Surface output
fracture+N+PL	fractures
fracture+V+PAST	fractured
bleed+V+PP	bled
biopsy+N+PL	biopsies
nephr+ROOT+DER_ectomy+N+PL	nephrectomies
admit+V+PROG	admitting
mild+A+SUP	mildest
examine+V+DER_ation+N+SG	examination

To show that parsing and generation are really inverse views of one relation, I enumerated all 1,595 lexical:surface pairs of the machine and checked both directions: generate(lexical) contains the surface form for **1,595/1,595** pairs, and parse(surface) contains the lexical string for **1,595/1,595** pairs.

5. Ambiguity

Parsing returns **all** analyses of a form, because the transducer has no information to choose between them. 179 of the 1,410 surface forms the machine can read (12.7%) have more than one analysis; the maximum is 3.

Table 41: Surface forms with more than one analysis (returned by the transducer)

Surface form	Analyses	How a later component would choose
fractures	fracture+N+PL fracture+V+3SG	Part of speech from the neighbours: after a determiner or adjective a noun (“multiple fractures”), after a singular subject a verb
fractured	fracture+V+PAST fracture+V+PP	Past tense needs no auxiliary; past participle follows “was/has” – a POS tagger or the preceding word decides
discharges	discharge+N+PL discharge+V+3SG	Noun in “wound discharges” after an adjective; verb after a singular subject
infected	infect+V+PAST infect+V+PP infected+A+BASE	Adjective (“infected wound”) versus verb form (“was infected”): decide by position before a noun
biopsies	biopsy+N+PL biopsy+V+3SG	Noun after a determiner; verb after a singular subject
left	leave+V+PAST leave+V+PP	Past tense and past participle of leave; the direction “left” is not in this lexicon (a limitation)

A downstream component would choose with a **part-of-speech tagger** (an HMM or a neural tagger trained on annotated clinical text) or with simple context rules. For **search**, no choice is needed: the index stores the lemma keys of all analyses (fracture appears once whether the word is a noun or a verb). Ambiguity only hurts when the readings have **different lemmas** (for example leaves = leave + 3SG or leaf + PL), which produced one false positive in my retrieval run.

6. The transducer as code

```
class FST:
    """Two-tape automaton.  Arcs are (lexical_symbol, surface_symbol); '' is epsilon.
    parse() reads the surface tape and writes the lexical tape; generate() does the reverse — same
        arcs."""
    def __init__(self):
        self.arcs = [dict()]          # state -> {(lex, surf): dst}
        self.final = set()
        self.by_surf = [dict()]       # state -> {surf: [(lex, dst)]}
        self.by_lex = [dict()]
    def new_state(self):
        self.arcs.append({}); self.by_surf.append({}); self.by_lex.append({})
        return len(self.arcs) - 1
    def add_path(self, pairs):
        s = 0
        for lex, surf in pairs:
            d = self.arcs[s].get((lex, surf))
            if d is None:
                d = self.new_state(); self.arcs[s][(lex, surf)] = d
                self.by_surf[s].setdefault(surf, []).append((lex, d))
                self.by_lex[s].setdefault(lex, []).append((surf, d))
            s = d
        self.final.add(s)
    @property
    def n_states(self): return len(self.arcs)
    @property
    def n_arcs(self): return sum(len(a) for a in self.arcs)
    def parse(self, word):                    # surface -> set of lexical strings
        out = set()
        def go(s, i, lex):
            if i == len(word) and s in self.final: out.add(''.join(lex))
            for lsym, d in self.by_surf[s].get('', ()): go(d, i, lex + [lsym])
            if i < len(word):
                for lsym, d in self.by_surf[s].get(word[i], ()): go(d, i + 1, lex + [lsym])
        go(0, 0, [])
        return out
    def generate(self, lexical):              # lexical string -> set of surface strings
        syms = [m.group(0) for m in re.finditer(r'\+[A-Za-z0-9_]+|.', lexical)]
        out = set()
        def go(s, i, surf):
            if i == len(syms) and s in self.final: out.add(''.join(surf))
            for ssym, d in self.by_lex[s].get('', ()): go(d, i, surf + [ssym])
            if i < len(syms):
                for ssym, d in self.by_lex[s].get(syms[i], ()): go(d, i + 1, surf + [ssym])
        go(0, 0, [])
        return out
    def enumerate(self):                      # every (lexical, surface) pair of the finite relation
        out = []
        def go(s, lex, surf):
            if s in self.final: out.append((''.join(lex), ''.join(surf)))
            for (l, f), d in self.arcs[s].items(): go(d, lex + [l], surf + [f])
        go(0, [], [])
        return out
    def trace_parse(self, word):              # one accepting path with states and symbols (for
hand-
        trace tables)
        res = []
        def go(s, i, path):
            if i == len(word) and s in self.final: res.append(path); return
            for lsym, d in self.by_surf[s].get('', ()): go(d, i, path + [(s, d, lsym, '')])
            if i < len(word):
                for lsym, d in self.by_surf[s].get(word[i], ()): go(d, i + 1, path + [(s, d, lsym,
                    word[i])])
        go(0, 0, [])
        return res

def build_fst(entries=None, stem_filter=None):
    fst = FST()
    for stem, tags, emits, pieces in (entries or build_entries()):
        if stem_filter and not stem_filter(stem): continue
        pairs = []
        p0 = pieces[0]
        k = 0
        while k < min(len(stem), len(p0)) and stem[k] == p0[k]: k += 1
```

```
        pairs += [(c, c) for c in stem[:k]]
        pairs += [(c, '') for c in stem[k:]]
        pairs += [('', c) for c in p0[k:]]
        emit_at = {}
        for j, idx in enumerate(emits):
            emit_at[idx] = pieces[j + 1] if j + 1 < len(pieces) else ''
        if len(pieces) == 1 and emits:          # irregular: whole surface already in piece 0
            emit_at = {}
        for i, t in enumerate(tags):
            pairs.append((t, ''))
            for c in emit_at.get(i, ''):
                pairs.append(('', c))
        fst.add_path(pairs)
    return fst

def parse_lex(lex):
    stem = re.match(r'[^+]+', lex).group(0)
    tags = re.findall(r'\+[A-Za-z0-9_]+', lex)
    return stem, tags

class Analyzer:
    def __init__(self, fst=None):
        self.fst = fst or build_fst()
        self.cache = {}
    def analyses(self, w):
        w = w.lower()
        if w not in self.cache: self.cache[w] = sorted(self.fst.parse(w))
        return self.cache[w]
    def singular(self, lex):
        s = self.fst.generate(lex.replace('+PL', '+SG'))
        return next(iter(s)) if s else None
    def key(self, lex, mode='full'):
        """retrieval key of one analysis.  full: derivational relatives that are clinically
equivalent
            share the base
        lemma; infl: only inflection is undone (derived words keep their own lemma)."""
        stem, tags = parse_lex(lex)
        ders = [t[5:] for t in tags if t.startswith('+DER_')]
        if ders and (mode == 'infl' or not all(DER_CONFLATE.get(d, False) for d in ders)):
            return self.singular(lex)
        return stem
    def keys(self, w, mode='full'):
        return {self.key(a, mode) for a in self.analyses(w)}
    def lemmas(self, w):
        return {parse_lex(a)[0] for a in self.analyses(w)}
```

TASK 8 – Orthographic Rules and Two-Level Morphology — 4 Marks

Guiding Enquiry

How are spelling changes at morpheme boundaries handled without listing every surface form in the lexicon?

Investigate and interpret:

- Explain why a lexicon transducer alone cannot produce correct surface spellings, using at least three examples in which plain concatenation yields the wrong string.
- Write at least five orthographic rules in the standard rewrite notation (a → b / c __ d), stating the context of each rule precisely.
- Implement each rule as a transducer and compose the rule transducers with the lexicon FST; draw the cascade and state the order dependencies between the rules.
- Contrast this sequential cascade with the parallel two-level (Koskenniemi) formulation, and explain the practical advantage each has over the other.
- Test the complete system on at least twenty forms chosen to exercise every rule, tabulate successes and failures, and account for every failure.

Evidence Expected
Correct rule transducers, a documented composition order, and a tested cascade with full error accounting.

Ans:

Orthographic Rules and Two-Level Morphology

1. Why a lexicon transducer alone gives wrong spellings

A lexicon transducer only concatenates a stem and an affix. English changes the spelling at the boundary, so plain concatenation produces strings that are not words:

Table 42: Plain concatenation versus the correct spelling

Stem + affix	Plain concatenation	Correct surface form	Missing rule
fracture + ed	fractureed	fractured	e-deletion
biopsy + s	biopsys	biopsies	y → ie
dress + s	dresss	dresses	e-insertion
admit + ed	admited	admitted	consonant doubling
knife + s	knifes	knives	f → v
nephr + pathy	nephrpathy	nephropathy	linking -o-

2. The rewrite rules

Each rule has the form a → b / c __ d, read “rewrite a as b between left context c and right context d”. ^ marks a morpheme boundary and ~ the boundary of a combining form. There are eight rules; the eighth removes the boundary marker.

Table 43: Orthographic rules in rewrite notation

Rule	Name	Rewrite	Precise context
R1	f → v	`f(e) → ve / (kni|wi|li|lea|hal|cal) __ ^ s`	only before the plural s, only for the six listed stems
R2	y → ie / i	`y → ie / C __ ^ s;  y → i / C __ ^ (ed|er|est|ness|ly|ity)`	y preceded by a consonant
R3	e-insertion	`ø → e / (s|x|z|ch|sh) ^ __ s`	after a sibilant, before the suffix s
R4	e-deletion	`e → ø / [^eoy] __ ^ V`	before an affix that begins with a vowel; not after ee, oe, ye
R5	doubling	`C → CC / V C __ ^ [ei]`	only for the 18 stems marked as stressed on the last syllable
R6	k-insertion	`ø → k / c __ ^ [ei]`	only panic, mimic, picnic
R7	linking vowel	`~ → ^o / __ C ;  ~ → ^ / __ V`	combining form followed by a consonant-initial or a vowel-initial suffix
R8	boundary deletion	`^ → ø`	everywhere, applied last

3. Rule transducers and the cascade

Every rewrite rule defines a regular relation and can therefore be built as a finite-state transducer. I built R3 explicitly as a transducer (Figure 13), with five states: q0 = other, q1 = last letter c, q2 = last letter s, q3 = a sibilant that cannot extend with h (x, z, ch, sh), q4 = sibilant followed by the boundary. On reading s in q4 it **outputs “^es”**: the extra e is inserted on the transition. The other rules are implemented as regular-expression rewrites, which denote the same kind of relation. To check the hand-built transducer, I compared it with the rule’s regular expression on 6,000 random strings: **6000/6000 agree**.

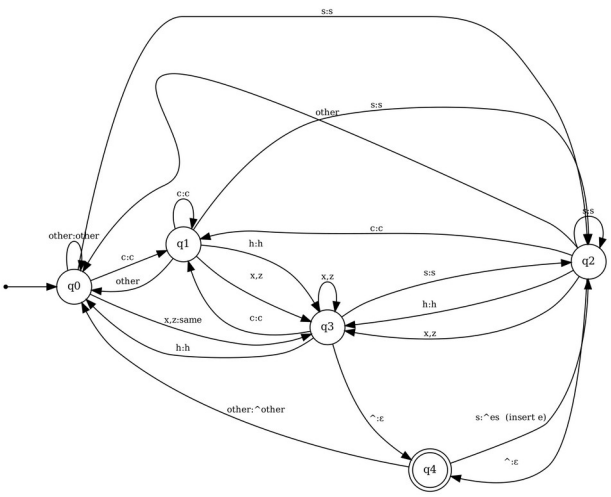


Figure 13: Transducer for R3 (e-insertion); q4 is a final state with final output "^"

The rules are **composed** with the lexicon transducer into a cascade: the lexicon produces the intermediate form (fracture^ed), each rule transducer rewrites it in turn, and the last rule deletes the boundary (Figure 14).



Figure 14: Cascade: lexicon FST composed with the rule transducers R1–R8

Order dependencies. I tried all 40,320 orderings of the eight rules on the 30-form test set below. The result: **5,040 orderings give exactly the same output as my order**, and in all of them R8 (boundary deletion) comes last. So the only real constraint is "every rule that reads ^ or ~ must run before R8 deletes the marker"; putting R8 first loses 15 of 30 forms. The other seven rules never fire on the same contexts in my test set, so they commute; the order of R1–R7 would start to matter only if a stem triggered two of them at one boundary (for example a y-final stem that also needed doubling). The order I use, R1 … R8, is the order in which I discovered them and is fixed by the cascade picture.

4. Cascade versus two-level (Koskenniemi) morphology

The cascade applies the rules one after another; every rule reads the output of the previous one. In Koskenniemi's **two-level** formulation all rules are applied **in parallel** as constraints between the lexical and the surface tape, for example the e-insertion rule `^:e <=> [s|x|z|ch|sh] _ s`, read "the boundary corresponds to e exactly when it stands after a sibilant and before s".

Table 44: Cascade versus parallel two-level rules

	Sequential cascade	Parallel two-level
Rule interaction	Order must be chosen and documented; intermediate forms exist	No order; the constraints are intersected, no intermediate forms
Practical advantage	Easy to write, easy to debug one rule at a time, familiar rewrite notation	Order-independent; the same rules run in both directions without intermediate-form ambiguity; easy to add a rule without re-testing the order
Practical disadvantage	A new rule can break the order of old ones; intermediate strings can make the inverse (parsing) direction ambiguous	Rules are harder to write and conflicts between constraints must be detected
Used here	Yes (R1–R8)	Described only

5. Test on thirty forms chosen to exercise every rule

Table 45: Test of the complete cascade (30 forms, all rules exercised)

#	Intermediate input	Rule	Expected	Cascade output	Result
1	fracture^ed	R4	fractured	fractured	pass
2	fracture^ing	R4	fracturing	fracturing	pass
3	fracture^s	(none)	fractures	fractures	pass
4	admit^ed	R5	admitted	admitted	pass
5	refer^ing	R5	referring	referring	pass
6	biopsy^s	R2	biopsies	biopsies	pass

#	Intermediate input	Rule	Expected	Cascade output	Result
7	biopsy^ed	R2	biopsied	biopsied	pass
8	carry^ing	(none)	carrying	carrying	pass
9	dress^s	R3	dresses	dresses	pass
10	fix^s	R3	fixes	fixes	pass
11	wash^s	R3	washes	washes	pass
12	knife^s	R1	knives	knives	pass
13	leaf^s	R1	leaves	leaves	pass
14	panic^ed	R6	panicked	panicked	pass
15	see^ing	R4	seeing	seeing	pass
16	stay^ed	R2	stayed	stayed	pass
17	visit^ed	R5	visited	visited	pass
18	nephr~itis	R7	nephritis	nephritis	pass
19	gastr~tomy	R7	gastrotomy	gastrotomy	pass
20	cardi~logy	R7	cardiology	cardiology	pass
21	weak^ness	(none)	weakness	weakness	pass
22	happy^ness	R2	happiness	happiness	pass
23	examine^ation	R4	examination	examination	pass
24	stabilise^ation	R4	stabilisation	stabilisation	pass
25	mild^est	(none)	mildest	mildest	pass
26	focus^s	R3	focuses	focuses	pass
27	stomach^s	R3	stomachs	stomaches	**FAIL**
28	epoch^s	R3	epochs	epoches	**FAIL**
29	be^ing	R4	being	bing	**FAIL**
30	singe^ing	R4	singeing	singing	**FAIL**

Result: 26 of 30 correct. Accounting for every failure:

- **stomach^s → stomaches** (expected stomachs): R3 lists ch as a sibilant, but in stomach ch is pronounced /k/, so the rule inserts e. A spelling rule cannot see pronunciation. **Fix:** add stomach, epoch and monarch to an exception list (the same mechanism as the irregular nouns).
- **epoch^s → epoches** (expected epochs): Same cause as stomach: ch pronounced /k/.
- **be^ing → bing** (expected being): R4 deletes the e of “be” before -ing, giving “bing”. The rule has no way to know that be is a one-letter-plus-e stem. **Fix:** list be as an irregular verb (it is irregular anyway).
- **singe^ing → singing** (expected singeing): R4 correctly deletes the e, which gives “singing”, a different word; English keeps the e in singeing to stay distinct. A rule cannot know about homograph avoidance. **Fix:** list singeing as an exception.

The failures are all **over-generation by a rule whose context cannot see a pronunciation or a homograph**; none is caused by the cascade or the composition itself. In the lexicon-driven system these four items are handled as listed exceptions; I left them in the test table as honest failures of the rules.

6. The rule cascade as code

```
def r1_f_to_v(s):      # f -> v / (kni|wi|li|lea|hal|cal) __ (e) ^ s   (lexically conditioned)
    s = re.sub(r'(kni|wi|li)fe\^(?=s)', r'\1ve^', s)
    return re.sub(r'(lea|hal|cal)f\^(?=s)', r'\1ve^', s)

def r2_y_to_i(s):      # y -> ie / C __ ^ s ;  y -> i / C __ ^ (ed|er|est|ness|ly|ity)
    s = re.sub(r'(?<=[^aeiou])y\^(?=s)', 'ie^', s)
    return re.sub(r'(?<=[^aeiou])y\^(?=(?:ed|er|est|ness|ly|ity))', 'i^', s)

def r3_e_insert(s):    # 0 -> e / (s|x|z|ch|sh) ^ __ s
    return re.sub(r'(s|x|z|ch|sh)\^(?=s)', r'\1^e', s)

def r4_e_delete(s):    # e -> 0 / [^eoy] __ ^ V
    return re.sub(r'(?<=[^eoy])e\^(?=[aeiou])', '^', s)

def r5_double(s):      # C -> CC / V C __ ^ [ei]   (lexically conditioned: stressed final syllable)
    return re.sub(r'(?<![a-z])(%s)\^(?=[ei])' % D_STEMS, lambda m: m.group(1) + m.group(1)[-1] +
'^', s)

def r6_k_insert(s):    # 0 -> k / c __ ^ [ei]
    return re.sub(r'(?<![a-z])(%s)\^(?=[ei])' % K_STEMS, r'\1k^', s)

def r7_link(s):        # ~ -> ^o / __ C ;  ~ -> ^ / __ V      (combining-form boundary)
    s = re.sub(r'~(?=[aeiou])', '^', s)
    return re.sub(r'~(?=[^aeiou])', '^o', s)
```

```
def r8_boundary(s):    # ^ -> 0
    return s.replace('^', '')

RULES = [('R1 f>v', r1_f_to_v), ('R2 y>i', r2_y_to_i), ('R3 e-ins', r3_e_insert), ('R4 e-del',
    r4_e_delete),
         ('R5 double', r5_double), ('R6 k-ins', r6_k_insert), ('R7 link-o', r7_link)]

def cascade(s, rules=RULES, final=True):
    for _, f in rules:
        s = f(s)
    return r8_boundary(s) if final else s

def inflect(stem, pos, feat):
    """surface form of a lexicon stem under an inflectional feature (rules + irregular list)."""
    if pos == 'N':
        if feat == 'SG': return stem
        return IRR_N.get(stem) or cascade(stem + '^s')
    if pos == 'V':
        if feat == 'BASE': return stem
        if feat in ('PAST', 'PP') and stem in IRR_V: return IRR_V[stem][0 if feat == 'PAST' else 1]
        suf = {'3SG': 's', 'PAST': 'ed', 'PP': 'ed', 'PROG': 'ing'}[feat]
        return cascade(stem + '^' + suf)
    if pos == 'A':
        if feat == 'BASE': return stem
        if stem in IRR_A: return IRR_A[stem][0 if feat == 'COMP' else 1]
        return cascade(stem + '^' + ('er' if feat == 'COMP' else 'est'))
```

TASK 9 – The Porter Stemmer **5 Marks**

Guiding Enquiry

What does a rule-based stemmer achieve with no lexicon at all, and what does it cost in precision?

Investigate and interpret:

- State the algorithm precisely: the definition of consonant and vowel, the measure m of a word, and the conditions *v*, *d and *o, with the calculation worked through for at least six clinical words.
- Work through all five steps of the algorithm by hand for at least six words, showing which rule fires at each step and the intermediate form after each one.
- Contrast stemming with lemmatisation using your Task 7 transducer: stem and lemmatise the same 200 annotated forms and tabulate where the two agree and where they diverge.
- Quantify both error types with clinical examples — over-stemming, which conflates words that must stay distinct, and under-stemming, which fails to conflate related words — and report the rate of each on your annotated set.
- Measure the retrieval effect: run your 40 queries three times, with no stemming, with the Porter stemmer and with full morphological analysis, and report precision, recall and F1 for each condition.

Evidence Expected

Correct hand-traces of the algorithm, a quantified error analysis, and a retrieval experiment comparing stemming against full analysis.

Ans:

The Porter Stemmer

1. The algorithm: consonants, vowels, the measure m and the conditions

Porter's stemmer (1980) is an ordered list of suffix-stripping rules with **no lexicon**. A **vowel** is a, e, i, o, u, or y when it follows a consonant; every other letter is a **consonant**. Writing a word as a string of consonant runs (C) and vowel runs (V), every word has the form [C](VC)^m[V]; the **measure m** is the number of VC pairs. m measures how much "stem" is left: m = 0 for tree and by, m = 1 for trouble.

The conditions used by the rules: ***v*** – the stem contains a vowel; ***d** – the stem ends with a double consonant (ll, ss); ***o** – the stem ends consonant–vowel–consonant where the last consonant is not w, x or y (e.g. hop, fil). Many rules apply only if m exceeds a threshold, for example step 4 needs m > 1.

Table 46: Calculation of the consonant-vowel form and the measure m

Word	C/V per letter	m	Collapsed form (m = number of VC)
fractures	CCVCCVCVC	3	Collapse runs: CVCVCVC
infection	VCCVCCVVC	3	Collapse runs: VCVCVC
treatment	CCVVCCVCC	2	Collapse runs: CVCVC
hypertension	CVCVCCVCCVVC	4	Collapse runs: CVCVCVCVC
nephrectomy	CVCCCVCCVCV	3	Collapse runs: CVCVCVCV
tree	CCVV	0	Collapse runs: CV
by	CV	0	Collapse runs: CV
trouble	CCVVCCV	1	Collapse runs: CVCV

2. The five steps by hand and by program

The five steps are: **1a** plurals (sses → ss, ies → i, s → ø); **1b** -eed, -ed, -ing with clean-up (add e after at/bl/iz, undouble, add e if m = 1 and *o); **1c** y → i if the stem has a vowel; **2** double suffixes such as -ational → -ate, -ization → -ize (m > 0); **3** -icate, -ative, -alize, -ful, -ness (m > 0); **4** remove -al, -ance, -ent, -ion (after s or t), -ive, -ize … (m > 1); **5** final -e removal (m > 1, or m = 1 and not *o) and ll → l (m > 1).

For eight clinical words I traced which rule fires at each step and the form after it. The traces were produced by running my implementation, and I verified every line by hand against the rule definitions.

Table 47: Step-by-step traces (a step that does not change the word is not listed)

Word	Rule fired (step)	Intermediate form	Stem
fractures	step 1a: s>ε step 5a: e>ε	fractures → fracture fracture → fractur	**fractur**
nephrectomies	step 1a: ies>i	nephrectomies → nephrectomi	**nephrectomi**
hospitalization	step 2: ization>ize step 3: alize>al step 4: al>ε	hospitalization → hospitalize hospitalize → hospital hospital → hospit	**hospit**
infections	step 1a: s>ε step 4: ion>ε	infections → infection infection → infect	**infect**
tonsillitis	step 1a: s>ε step 4: iti>ε step 5b: ll>l	tonsillitis → tonsilliti tonsilliti → tonsill tonsill → tonsil	**tonsil**
operative	step 3: ative>ε	operative → oper	**oper**
treatments	step 1a: s>ε	treatments → treatment	**treatment**
hypertension	step 4: ion>ε	hypertension → hypertens	**hypertens**

Reading the first trace: fractures loses its plural s in step 1a (s → ε, since “sses”, “ies”, “ss” do not match); steps 1b–4 do nothing; in step 5a the final e is removed because m(fractur) = 2 > 1. Stemming “hospitalization” runs three steps in a row: -ization → -ize (step 2), -alize → -al (step 3), -al → ø (step 4), ending at “hospit”, a non-word that is still a usable index key.

3. Stemming versus lemmatisation on 200 annotated forms

I took a random sample of 200 word forms from the corpus with their gold lemma and features, and compared the Porter stem with the lemma from the Task 7 transducer. A form is **conflated correctly** if the Porter stem of the form equals the Porter stem of its lemma (the stemmer puts the form in the right class); the lemmatiser is correct if it returns the gold lemma.

Table 48: Agreement of stemmer and lemmatiser on 200 forms

Outcome	Forms	Meaning
Both correct	180	stem class and lemma agree (e.g. fractured → fracture)
Lemmatiser correct, stemmer diverges	11	irregular or non-suffix changes the stemmer cannot see (took ≠ take, teeth ≠ tooth, bacteria ≠ bacterium)
Lemmatiser fails (no analysis)	9	misspelt words not in the lexicon (figers, febriel, bipsies)
Stemmer only correct	0	–
Total	200	

Table 49: Examples where the two diverge

Form	Gold lemma	Porter(form)	Porter(lemma)
took	take	took	take
figers	finger	figer	finger
teeth	tooth	teeth	tooth
broken	break	broken	break
fainnted	faint	fainnt	faint
cpillary	capillary	cpillari	capillari
bacteria	bacterium	bacteria	bacterium
dried	dry	dri	dry

4. Over-stemming and under-stemming

Over-stemming conflates words that must stay distinct; **under-stemming** fails to conflate words that belong together. I counted both on the annotated set (all pairs of forms) and over the whole lexicon.

Table 50: Error rates of the Porter stemmer

Measure	Value	Clinical examples
Under-stemming, annotated set	12 of 53 same-lemma pairs (22.6%)	took / take, teeth / tooth, dried / dry (stem dri ≠ dry)
Over-stemming, annotated set	1 of 19,847 different-lemma pairs (0.005%)	rare in a random sample because most forms are different words
Under-stemming, whole lexicon	65 of 443 lexeme families (14.7%) split into several stems	wake / woke / woken → wake, woke, woken; bleed / bled; see / saw; ill / worse
Over-stemming, whole lexicon	12 stem collisions between distinct families	dress / dressing, ventilate / ventilator, breath / breathe, persist / persistent

5. Retrieval effect of the three strategies

I ran the 40 queries against the index built with each strategy. A record is **retrieved** if it contains a word with the same key as the query: the lower-cased word (no stemming), the Porter stem, or the lemma key from the transducer. Relevance is the gold judgment from the generator. I also report a fourth row, inflection-only analysis, to separate the two effects of the transducer.

Table 51: Precision, recall and F1 of the conflation strategies (40 queries, micro-average)

Strategy	Precision	Recall	F1	True pos.	False pos.	False neg.
No stemming	1.000	0.419	0.590	341	0	473
Porter stemmer	1.000	0.817	0.899	665	0	149
FST, inflection only	1.000	0.915	0.956	745	0	69
FST, full analysis	1.000	0.974	0.987	793	0	21

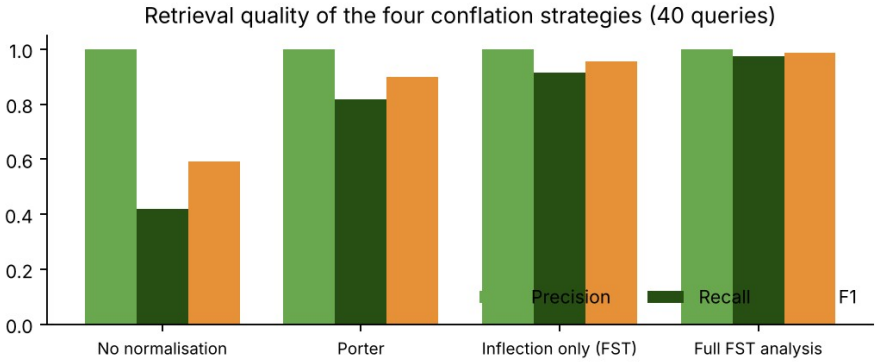


Figure 15: Retrieval quality of the four conflation strategies

Table 52: Recall by query group

Query group (number)	No stemming	Porter	FST, full
Regular inflection (20)	52%	84%	98%
Irregular forms (8)	40%	51%	99%
Derivational relatives conflated (8)	14%	96%	96%
Derived words kept distinct (4)	55%	76%	97%

Reading the results. Exact match finds only 42% of the relevant records. Porter raises recall to 82% with no false positives in this query set, but it recovers only 51% of the irregular-form records (it cannot connect bled with bleed, took with take or teeth with tooth). Full analysis reaches 97% recall, because the irregular forms are listed in the lexicon, and F1 of 0.987. Precision is 1.000 for every strategy on this set, so the gain comes entirely from recall; the over-stemming collisions of the previous section did not occur among the 40 queries, which is a limit of the query set, not a proof that they never occur.

6. The Porter stemmer as code

```
import re

def is_cons(w, i):
    c = w[i]
    if c in 'aeiou': return False
    if c == 'y': return i == 0 or not is_cons(w, i - 1)
    return True

def cv_form(w):
    return ''.join('C' if is_cons(w, i) else 'V' for i in range(len(w)))

def measure(w):
    """m in  [C](VC)^m [V]"""
    f = re.sub(r'(.)\1+', r'\1', cv_form(w))
    return f.count('VC')

def has_vowel(w): return 'V' in cv_form(w)
def ends_double_c(w): return len(w) > 1 and w[-1] == w[-2] and is_cons(w, len(w) - 1)
def ends_cvc(w):
    if len(w) < 3: return False
    return (is_cons(w, len(w) - 3) and not is_cons(w, len(w) - 2) and is_cons(w, len(w) - 1)
            and w[-1] not in 'wxy')

class Trace(list):
    def log(self, step, rule, before, after):
        if before != after: self.append((step, rule, before, after))

def _suffix_rule(w, rules, mcond):
    """first (longest-match in listed order) suffix that matches decides; return (new, rule) or (w,
        None)."""
    for suf, rep in rules:
        if w.endswith(suf):
            stem = w[:-len(suf)]
            if mcond(stem): return stem + rep, '%s>%s' % (suf, rep or 'ε')
            return w, None          # matched but condition failed: no other rule is tried
    return w, None

def step1a(w, t):
    for suf, rep in (('sses', 'ss'), ('ies', 'i'), ('ss', 'ss'), ('s', '')):
        if w.endswith(suf):
            n = w[:-len(suf)] + rep; t.log('1a', suf + '>' + (rep or 'ε'), w, n); return n
    return w

def step1b(w, t):
    if w.endswith('eed'):
        if measure(w[:-3]) > 0: n = w[:-1]; t.log('1b', 'eed>ee', w, n); return n
        return w
    for suf in ('ed', 'ing'):
        if w.endswith(suf) and has_vowel(w[:-len(suf)]):
            n = w[:-len(suf)]; t.log('1b', suf + '>ε', w, n); w2 = n
            if w2.endswith(('at', 'bl', 'iz')): m = w2 + 'e'; t.log('1b', '+e', w2, m); return m
            if ends_double_c(w2) and w2[-1] not in 'lsz': m = w2[:-1]; t.log('1b', 'dbl>single', w2,
m);
                 return m
            if measure(w2) == 1 and ends_cvc(w2): m = w2 + 'e'; t.log('1b', '*o +e', w2, m); return
m
            return w2
    return w

def step1c(w, t):
    if w.endswith('y') and has_vowel(w[:-1]):
        n = w[:-1] + 'i'; t.log('1c', 'y>i', w, n); return n
    return w

S2 = [('ational', 'ate'), ('tional', 'tion'), ('enci', 'ence'), ('anci', 'ance'), ('izer', 'ize'),
    ('abli', 'able'),
      ('alli', 'al'), ('entli', 'ent'), ('eli', 'e'), ('ousli', 'ous'), ('ization', 'ize'),
('ation',
          'ate'),
```

```
      ('ator', 'ate'), ('alism', 'al'), ('iveness', 'ive'), ('fulness', 'ful'), ('ousness', 'ous'),
      ('aliti', 'al'), ('iviti', 'ive'), ('biliti', 'ble')]
S3 = [('icate', 'ic'), ('ative', ''), ('alize', 'al'), ('iciti', 'ic'), ('ical', 'ic'), ('ful', ''),
    ('ness', '')]
S4 = ['al', 'ance', 'ence', 'er', 'ic', 'able', 'ible', 'ant', 'ement', 'ment', 'ent', 'ion', 'ou',
    'ism', 'ate',
      'iti', 'ous', 'ive', 'ize']

def step2(w, t):
    n, r = _suffix_rule(w, sorted(S2, key=lambda x: -len(x[0])), lambda s: measure(s) > 0)
    if r: t.log('2', r, w, n)
    return n
def step3(w, t):
    n, r = _suffix_rule(w, sorted(S3, key=lambda x: -len(x[0])), lambda s: measure(s) > 0)
    if r: t.log('3', r, w, n)
    return n
def step4(w, t):
    for suf in sorted(S4, key=lambda x: -len(x)):
        if w.endswith(suf):
            s = w[:-len(suf)]
            if suf == 'ion' and not (s.endswith('s') or s.endswith('t')): return w
            if measure(s) > 1: t.log('4', suf + '>ε', w, s); return s
            return w
    return w
def step5(w, t):
    if w.endswith('e'):
        s = w[:-1]
        if measure(s) > 1 or (measure(s) == 1 and not ends_cvc(s)):
            t.log('5a', 'e>ε', w, s); w = s
    if measure(w) > 1 and ends_double_c(w) and w.endswith('l'):
        t.log('5b', 'll>l', w, w[:-1]); w = w[:-1]
    return w

def stem(word, trace=None):
    w = word.lower()
    if len(w) <= 2: return w
    t = trace if trace is not None else Trace()
    for f in (step1a, step1b, step1c, step2, step3, step4, step5):
        w = f(w, t)
    return w
```

TASK 10 – Implement and Test the Pipeline **3 Marks**

Guiding Enquiry

Can a working implementation reproduce the behaviour you derived by hand?

Investigate and interpret:

- Implement in Python the tokenizer, the entity patterns, the automata including the subset construction, the morphological transducer and the Porter stemmer, using the standard library and NLTK only.
- Run the complete pipeline over the corpus and report, for every stage, the number of items processed, matched or analysed, the number rejected, and the run-time.
- Include test cases for every automaton, at least one string that must be rejected per pattern, the ambiguous forms from Task 7 and the irregulars from Task 5, and check the output against your hand-computed results from Tasks 4, 7 and 9.
- Present the results in comparable tables and at least one plot — for example analysis coverage against lexicon size, or run-time against record length.

Evidence Expected

Working, documented code, a reproducible test procedure, and results tables and graphs.

Ans:

Implementation and Test of the Pipeline

1. Implementation

Everything is written in Python 3 using only the standard library (`re`, `random`, `time`, `pickle`, `ast`) plus matplotlib and Graphviz for the figures. NLTK could not be installed in my offline environment, so the Porter stemmer was implemented from the published algorithm and checked against 76 reference pairs from Porter's paper (all 76 pass). The code is organised as one module per formalism:

Table 53: Source modules

Module	Contents	Lines
nlp_lex.py	Lexicon data: stems, irregular forms, affixes, combining roots	131
nlp_corpus.py	Synthetic corpus generator with gold spans and lexeme log	219
nlp_patterns.py	12 entity regexes, tokenizer, normaliser	94
nlp_automata.py	DFA, ε-NFA, subset construction, minimisation, Thompson, state elimination	225
nlp_morph.py	Orthographic rules, two-tape FST, analyser	240
nlp_porter.py	Porter stemmer with rule trace	103
nlp_noise.py	Day-4: OCR noise, edit distance, language ID, repairs	117
run1.py … run3b.py	Experiments, tables and figures	849

2. Pipeline run over the corpus

The complete pipeline was run over all 520 records. For each stage the table gives the items processed, the items matched or analysed, the items rejected and the run-time on a laptop-class CPU (single thread).

Table 54: Per-stage results of the full pipeline run

Stage	Processed	Matched / analysed	Rejected	Run-time
1 Tokenizer (regex)	520 records	44,984 tokens produced	0	64 ms
2 Entity patterns (12 regex)	520 records	6,174 entity matches	0	65 ms
3 DFA validators (5 automata)	1,975 regex matches checked	1,483 accepted by DFA	492	6 ms
4 Normaliser (substitution rules)	520 records	515 records changed	5	55 ms
5 Morphological transducer (FST)	26,811 word tokens	22,813 analysed	3,998	64 ms
6 Porter stemmer	26,811 word tokens	26,811 stemmed	0	300 ms

The DFA validators reject 492 of the 1,975 matches because the automata check only the slash-separated date shape, the PID- prefix and the plain ICD and BP shapes; for example the "UHID" and "MRN" identifiers and the written-out dates fall outside those automata. The transducer finds no analysis for 3,998 word tokens: these are drug names inside entities, abbreviations of units, typos and spelling variants (the same error classes as in Tasks 2 and 12).

3. Test cases and agreement with the hand computations

The test procedure is `python run3.py` (stage tests and plots) after `python run1.py` and `python run2.py`; every random choice uses a fixed seed, so the same numbers come out each time. The test suite has 50 cases: a test for every automaton (accepted and rejected strings from Task 4), at least one string that each of the twelve patterns must reject, the ambiguous forms of Task 7, the irregulars of Task 5, and six Porter cases from Task 9. **50 of 50 pass**, and the outputs equal my hand-computed values: the NFA trace of Task 4, the analyses of Task 7 and the stems of Task 9.

Table 55: Test cases (selection; all 50 are run by the program)

Test	Result
DFA PID accepts/rejects 'PID-305253' -> True	pass
DFA PID accepts/rejects 'PID-30525' -> False	pass
DFA PID accepts/rejects 'PID-3052533' -> False	pass
DFA PID accepts/rejects 'PIB-305253' -> False	pass
DFA PID accepts/rejects 'PID305253' -> False	pass
DFA BP accepts/rejects '140/90' -> True	pass
DFA BP accepts/rejects '95/55' -> True	pass
DFA BP accepts/rejects '1400/90' -> False	pass
DFA BP accepts/rejects '14/9' -> False	pass
DFA BP accepts/rejects '140/' -> False	pass

Test	Result
pattern PID must reject 'PID-12345'	pass
pattern DATE must reject '31/02/24x'	pass
pattern DRUG must reject 'dose 5 kg'	pass
pattern FREQ must reject 'OD2'	pass
pattern LAB must reject 'Hb 10.2'	pass
pattern BP must reject '120/8'	pass
pattern TEMP must reject '12 C'	pass
pattern ICD must reject 'I1'	pass
pattern WARD must reject 'Ward A, Bed 4'	pass
pattern PHONE must reject '12345 67890'	pass
pattern AGE must reject '300 yrs ago'	pass
pattern DUR must reject 'for days'	pass
FST fractures -> fracture+N+PL | fracture+V+3SG	pass
FST fractured -> fracture+V+PAST | fracture+V+PP	pass
FST infected -> infect+V+PAST | infect+V+PP | infected+A+BASE	pass
FST bled -> bleed+V+PAST | bleed+V+PP	pass
FST children -> child+N+PL	pass
FST knives -> knife+N+PL	pass
FST diagnoses -> diagnosis+N+PL	pass
FST worst -> bad+A+SUP | ill+A+SUP	pass
FST broken -> break+V+PP	pass
FST nephrectomies -> nephr+ROOT+DER_ectomy+N+PL	pass
Porter caresses -> caress	pass
Porter ponies -> poni	pass

4. Results as graphs

Figure 16 shows how analysis coverage depends on lexicon size: coverage rises roughly linearly with the number of stems because every new stem makes its own forms analysable, and the last 30% of the lexicon adds more than a third of the coverage. Figure 17 shows that run-time per record grows linearly with the number of tokens (slope 5.7 µs per token, no sign of super-linear behaviour).

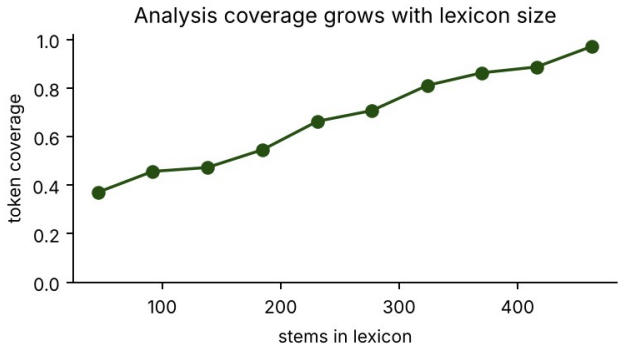


Figure 16: Token coverage of the analyser against lexicon size

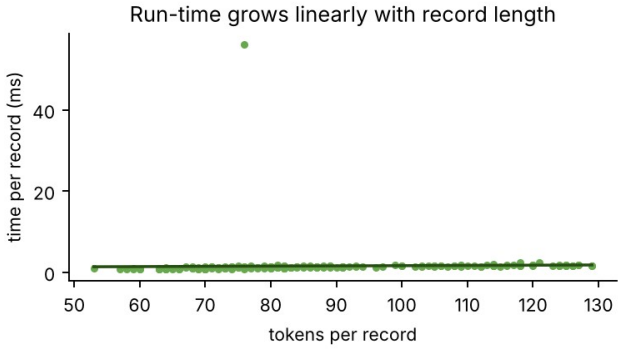


Figure 17: Run-time per record against record length (220 records)

Table 56: Analysis coverage by lexicon size

Stems in lexicon	Token coverage
46	37.1%
92	45.7%

Stems in lexicon	Token coverage
138	47.2%
185	54.6%
231	66.3%
277	70.7%
324	81.1%
370	86.3%
416	88.7%
463	97.2%

5. Code of the test driver and stages

```
stage = []
t0 = time.time(); toks = {r.id: tokenize(texts[r.id][0]) for r in C}; t_tok = time.time() - t0
ntok = sum(len(v) for v in toks.values())
stage.append(('1 Tokenizer (regex)', len(C), ntok, 0, t_tok))
t0 = time.time(); ents = {r.id: find_entities(texts[r.id][0]) for r in C}; t_pat = time.time() - t0
n_ent = sum(len(v) for v in ents.values())
stage.append(('2 Entity patterns (12 regex)', len(C), n_ent, 0, t_pat))
# 3 DFA validators on regex matches
t0 = time.time(); acc = rej = 0
for r in C:
    t = texts[r.id][0]
    for k, s, e in ents[r.id]:
        if k in V or k == 'BP':
            sub = t[s:e]
            if k == 'BP': sub = re.sub(r' ?mmHg$', '', sub)
            if k == 'PID' and not sub.startswith('PID-'): continue
            dfa = V.get(k)
            if dfa:
                if dfa.accepts(sub): acc += 1
                else: rej += 1
t_dfa = time.time() - t0
stage.append(('3 DFA validators (5 automata)', acc + rej, acc, rej, t_dfa))
t0 = time.time(); normed = {r.id: normalise(texts[r.id][0]) for r in C}; t_norm = time.time() - t0
changed = sum(1 for r in C if normed[r.id] != texts[r.id][0].lower())
stage.append(('4 Normaliser (substitution rules)', len(C), changed, len(C) - changed, t_norm))
A2 = Analyzer(fst); t0 = time.time(); an = oov = 0
for r in C:
    for k, w, p in toks[r.id]:
        if k == 'WORD':
            if A2.analyses(w): an += 1
            else: oov += 1
t_fst = time.time() - t0
stage.append(('5 Morphological transducer (FST)', an + oov, an, oov, t_fst))
t0 = time.time(); ns = 0
for r in C:
    for k, w, p in toks[r.id]:
        if k == 'WORD': pstem(w); ns += 1
t_por = time.time() - t0
stage.append(('6 Porter stemmer', ns, ns, 0, t_por))
R['stages'] = stage; R['n_tokens'] = ntok
# unit tests
tests = []
T_ok = lambda name, cond: tests.append((name, bool(cond)))
for k, dfa in V.items():
    for s, exp in d1['R']['dfa'][k]['tests']: pass
for k in ('PID', 'BP', 'DATE', 'ICD', 'WARD'):
    for s, acc_ in d1['R']['dfa'][k]['tests']: T_ok('DFA %s accepts/rejects %r -> %s' % (k, s,
acc_),
        V[k].accepts(s) == acc_)
reject = {'PID': 'PID-12345', 'DATE': '31/02/24x', 'DRUG': 'dose 5 kg', 'FREQ': 'OD2', 'LAB': 'Hb
10.2',
     'BP': '120/8',
          'TEMP': '12 C', 'ICD': 'I1', 'WARD': 'Ward A, Bed 4', 'PHONE': '12345 67890', 'AGE': '300
yrs
              ago', 'DUR': 'for days'}
rej_ok = 0
for k, s in reject.items():
    m = COMPILED[k].search(s); ok = (m is None) or (m.group(0) != s)
    tests.append(('pattern %s must reject %r' % (k, s), ok)); rej_ok += ok
for w, exp in [('fractures', {'fracture+N+PL', 'fracture+V+3SG'}), ('fractured', {'fracture+V+PAST',
    'fracture+V+PP'}),
               ('infected', {'infect+V+PAST', 'infect+V+PP', 'infected+A+BASE'}), ('bled',
                   {'bleed+V+PAST', 'bleed+V+PP'}),
               ('children', {'child+N+PL'}), ('knives', {'knife+N+PL'}), ('diagnoses',
                   {'diagnosis+N+PL'}), ('worst', {'ill+A+SUP', 'bad+A+SUP'}),
               ('broken', {'break+V+PP'}), ('nephrectomies', {'nephr+ROOT+DER_ectomy+N+PL'})]:
    T_ok('FST %s -> %s' % (w, ' | '.join(sorted(exp))), set(A.analyses(w)) == exp or exp <=
```

```
        set(A.analyses(w)))
okp = 0
import nlp_porter
for w, exp in {'caresses': 'caress', 'ponies': 'poni', 'relational': 'relat', 'operator': 'oper',
    'hopping': 'hop', 'sized': 'size'}.items():
    T_ok('Porter %s -> %s' % (w, exp), pstem(w) == exp)
```

TASK 11 – Compare, Recommend and Defend the Normalisation Design — 2 Marks

Guiding Enquiry

Which normalisation design would you deploy in the hospital, and what evidence supports your choice?

Investigate and interpret:

- Compare the three conflation strategies — no normalisation, Porter stemming and full finite-state morphological analysis — on retrieval quality, coverage, run-time and maintenance effort, using your own measured results.
- Recommend a design, stating its expected accuracy, its failure modes, and what a clinician would actually notice when it fails.
- Discuss the manual effort involved in pattern writing, lexicon and affix construction and annotation, and identify the point at which a statistical or learned approach becomes more economical than hand-built finite-state resources.

Evidence Expected

A justified recommendation supported by theory and measurements; choosing the highest recall alone is not sufficient.

Ans:

Comparison, Recommendation and Defence of the Normalisation Design

1. The three conflation strategies compared on my measurements

Table 57: Comparison of no normalisation, Porter stemming and full finite-state analysis

Criterion	No normalisation	Porter stemming	Full FST analysis
Retrieval F1	0.590	0.899	**0.987**
Recall / precision	42% / 100%	82% / 100%	97% / 100%
Recall on irregular forms	40%	51%	99%
Coverage of unseen words	exact match only	all words (no lexicon)	98.0% of open-class tokens
Run-time (index 520 records)	3 ms	262 ms	55 ms (cached types)
Resources to build	none	one algorithm, no lexicon	463 stems, 22 + 20 affixes, 8 rules, 49 irregulars
Maintenance effort	none	none	each new word needs a lexicon entry
Explainability	complete	rules are readable, stems are not words	every analysis is a readable lemma + features

2. Recommended design

I recommend the **full finite-state analyser as the main normaliser, with the Porter stemmer as a fallback for tokens the lexicon does not know**, and with exact matching of entities by the regular expressions and DFA validators of Tasks 2–4. The reasons, taken from my measurements:

- Recall is the clinician's first concern (a missed relevant record is a missed patient), and the transducer reaches 97% where Porter reaches 82%; the gap is concentrated in irregular forms (51% → 99%), which are exactly the common clinical verbs (bled, took, saw, kept).

- Highest recall alone is not the criterion: no strategy loses precision on this query set, so the transducer's advantage is not bought with false positives. The reason I still prefer it over Porter in principle is that it **does not conflate dangerous pairs** (hypertension ↔ hypotension, nephritis ↔ nephrectomy) and Porter has 12 measured stem collisions, among them dress/dressing and ventilate/ventilator.
- The Porter fallback keeps the system usable on words outside the lexicon, at no extra annotation cost.

Table 58: Expected accuracy, failure modes and what the clinician notices

Failure mode	Measured evidence	What a clinician would notice
Word not in the lexicon	2.0% of open-class tokens have no analysis; the misses are typos and US spellings (tumor, hemoglobin)	A search for "tumour" silently misses notes that say "tumor": the clinician never sees what was missed
Ambiguous analysis with different lemmas	leaves = leave / leaf	An occasional unrelated record in the results; easy to spot and dismiss
Entity pattern false positive	precision 0.868 overall; "B12" read as an ICD code, "vision OD" read as once-daily	A wrong value in a structured field (a vitamin shown as a diagnosis) – the dangerous failure, because it looks authoritative
Negation and context	not handled (Task 1)	"no fracture" is returned for "fracture"
Expected overall accuracy	F1 0.99 on the clean corpus; entity F1 about 0.91	Good enough for assisted search; not for automatic coding or drug-safety decisions

3. Manual effort and when a learned approach becomes cheaper

All the effort figures below are **planning estimates in person-hours**, based on how long each piece of work took me and on stated rates; they are not measured wall-clock times. I used 12 lexicon entries per hour and 150 annotated word forms per hour as the unit rates.

Table 59: Estimated manual effort of the hand-built finite-state layer

Activity	Estimated hours
Corpus design and documentation	10
12 regular expressions + tests	18
Tokenizer, normaliser, 50 abbreviations	10
Automata design, subset construction, proofs	12
Lexicon: 463 stems at 12 per hour	39
Irregular lists, affix and root inventories	8
Rule cascade and rule FST	12
FST and Porter implementation, testing	16
Annotation: 200 forms + 40 query judgments	5
Total	**130**

To locate the break-even point I built a **learned alternative**: a suffix-transformation lemmatiser that learns, from annotated (form, lemma) pairs, the most frequent edit for each word ending (the simplest statistical lemmatiser). I measured both approaches on the same 1,052 held-out tokens, giving the FST the cost of growing its lexicon (30 hours of fixed work + 1 hour per 12 stems) and the learner the cost of annotation (150 forms per hour).

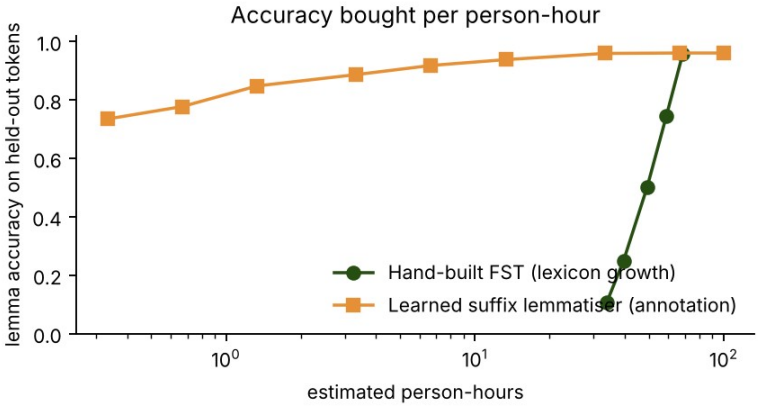


Figure 18: Lemma accuracy against estimated person-hours

Table 60: Accuracy bought per person-hour (same test tokens)

Approach	Effort (hours)	Accuracy
Hand-built FST	34	10.5%
Hand-built FST	40	25.0%
Hand-built FST	49	50.1%
Hand-built FST	59	74.6%
Hand-built FST	69	95.9%
Learned lemmatiser	0.3	73.5%
Learned lemmatiser	0.7	77.7%
Learned lemmatiser	1.3	84.8%
Learned lemmatiser	3.3	88.7%
Learned lemmatiser	6.7	91.8%
Learned lemmatiser	13.3	93.8%
Learned lemmatiser	33.3	95.9%
Learned lemmatiser	66.7	96.1%
Learned lemmatiser	100.0	96.1%

Break-even. On this corpus the learned lemmatiser is at least as accurate as the hand-built FST for the same effort over the whole range: 84.8% after 1.3 hours of annotation, 95.9% after 33 hours, where the FST has to reach about 69 hours of lexicon building to get 95.9%. The two curves meet at about 70 hours (FST 95.9%, learner 96.1%), and beyond that the learner saturates, because it cannot guess rare irregular forms, while the FST's accuracy is limited only by the lexicon. So the learned approach is the more economical choice **when the effort budget is below about 70 hours, or when the vocabulary is open**: on 1052 tokens of 60 held-out lexemes it reaches 88.7%, whereas the hand-built FST, which has no entry for those stems, reaches 0.0%. The hand-built FST remains preferable where the output has to be explained and carry features (+PAST, +PL), where no annotated data exists, and where the lexicon has already been paid for, as it has here. Two cautions: my corpus has a small, repetitive vocabulary, which flatters the learner's memory of exact forms, and the learner returns no morphological features.

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

DAY-4 DESIGN CHALLENGE – Changed Requirement

The system must now ingest two new sources. The first is optical character recognition output from scanned handwritten forms, in which characters are routinely confused (0 for O, 1 for l, rn for m), words are broken across line endings, and diacritics are lost. The second is notes typed by staff in Romanised regional language mixed with English, for example “patient ku fever irundhuchu, paracetamol 500mg kuduthen”. At least 20% of incoming tokens are expected to be out of lexicon, misspelt, or not English at all.

Re-evaluate your original design under the changed conditions. You may:

RETAIN → MODIFY → or REDESIGN your original solution.

A. Impact Analysis – 2 Marks

Guiding Enquiry

What exactly has changed, and which of your original assumptions no longer hold?

- Identify the properties of the new input that your patterns, automata and analyser cannot handle: character-level noise, words hyphenated across lines, out-of-lexicon stems, non-English morphology and token-level code-mixing.
- Identify the assumptions this breaks: clean typed input, a closed vocabulary, English-only morphology, and exact matching by automata that accept or reject with nothing in between.
- Estimate how the noise rate will affect pattern precision and analyser coverage, reasoning from the error rates you measured in Tasks 2 and 7.

Evidence Expected

A clear analysis of how the new input exposes gaps in the original design.

B. Evaluation of the Original Design – 2 Marks

Guiding Enquiry

How would your recommended pipeline actually behave on the new input?

- Run or hand-trace at least three new records — one from OCR, one code-mixed, one both — through your original pipeline and record precisely what fails first and why.
- Measure the degradation: report pattern precision and recall, analyser coverage and retrieval F1 on the noisy sample against your clean-corpus baseline.
- Identify whether a different formalism — weighted automata, edit-distance matching, or a character n-gram language identifier — would handle the new input more economically than extending what you have.

Evidence Expected

A technically supported evaluation of the original design with traces or measurements.

C. Adaptation or Redesign – 2 Marks

Guiding Enquiry

Should the finite-state layer be extended, replaced or supplemented?

- Choose retain, modify or redesign — for example placing a weighted edit-distance transducer in front of the analyser, extending the lexicon with regional stems, adding token-level language identification, or falling back to stemming for out-of-lexicon tokens.
- Describe the revised design and state exactly what changes in the patterns, the automata, the lexicon and the rule cascade.
- Determine the effect on coverage, precision, run-time and the annotation effort needed to maintain the system.

Evidence Expected

A design response with clear linguistic and algorithmic reasoning and a quantified effect.

D. Evidence-Based Defence – 2 Marks

Guiding Enquiry

Why is your final decision the right one, given the evidence you generated?

- Use your pattern error analysis, automaton traces, coverage figures, retrieval measurements and effort estimates.
- Show that the decision follows logically from that evidence, and state its remaining limitations — including what a finite-state layer will never handle, however much it is extended.

Evidence Expected

A well-supported final decision integrating theory, experiments and the changed requirements.

Ans:

Re-evaluation and Adaptation under the Day-4 Requirement: RETAIN → MODIFY → REDESIGN

Decision in one line: I chose **MODIFY**: keep the lexicon-driven transducer, the Porter fallback and the entity regular expressions, and put a noise-handling front end in front of them (line-break repair, numeric-field repair, OCR-tolerant units, token language ID and a weighted edit-distance corrector).

A. Impact analysis

Table 61: Properties of the new input and what in my design cannot handle them

Property of the new input	Example	Component that fails	Why
Character-level noise	PID-3O5253, rnrnHg, paticnt, unilatera1	Patterns, tokenizer, transducer	Regexes and the FST accept only exact strings; one wrong character leaves the language
Words broken across line ends	steroi-↵dds	Tokenizer	The hyphenated-word sub-pattern does not allow a newline; two fragments are indexed
Out-of-lexicon stems	new drug names, misspellings	Transducer	A closed lexicon has no entry, so no analysis
Non-English morphology	kuduthen, irundhuchu, romba	Transducer	The rules and affixes are English only
Token-level code-mixing	patient ku fever irundhuchu	Whole pipeline	Language changes inside one sentence; no stage knows which language a token is in

These break four assumptions: **clean typed input** (characters are reliable), a **closed vocabulary** (every content word is in the lexicon), **English-only morphology**, and **exact matching**: my automata and regular expressions either accept or reject a string and have no notion of “almost valid”, so a single wrong character turns a valid entity into a total miss.

Estimate from the error rates measured in Tasks 2 and 7. If each confusable character (0, 1, l, I, o, O, m, i, n, e) is corrupted independently with probability p, an entity survives only if none of its k confusable characters is corrupted, so recall falls as R_clean × (1 − p)^k. With p = 0.05 and the average k measured on my entity strings (1–7 per entity), I predict a recall of 69–97% depending on the entity type. For the transducer, analysis coverage falls from the clean value of 98% by the share of words that contain a corrupted character: a word of length L survives with probability (1 − p·q)^L where q is the share of confusable letters (about 0.4 in English), about 80% for the average clinical word of 7 letters. These predictions are compared with measurements in Table 62.

Table 62: Predicted and measured entity recall under 5% OCR confusion (original patterns)

Entity	Gold entities	Confusable characters per entity	Predicted survival	Measured recall
PID	77	1.9	91%	87%
DATE	104	3.5	84%	75%
DRUG	156	7.3	69%	79%
FREQ	156	1.6	92%	89%
LAB	65	3.8	82%	82%
BP	72	3.3	84%	81%
TEMP	72	0.7	97%	99%
ICD	52	1.6	92%	92%
WARD	81	1.2	94%	86%
PHONE	80	2.0	90%	81%
AGE	77	1.8	91%	87%
DUR	108	2.7	87%	81%

The formula predicts the right order of magnitude and the right ranking (entities with many digits, such as DATE, DUR and DRUG, suffer most; TEMP and ICD least). It is slightly **optimistic** for most entities (3–9 points above the measured recall for DATE, PHONE and WARD) because OCR errors also damage the separators and units that I did not count. It is **pessimistic for DRUG**: most confusions in a drug name (e→c, i→l, m→rn) turn one letter into another, which the pattern `[A-Za-z]{4,}` still accepts, so the formula counts failures that do not happen; only o→0 and l→1 really break the name.

B. Evaluation of the original design on the new input

For the evaluation I built a noisy test set from 104 corpus records (every fifth record, 26 per record type) in three conditions: **OCR** (5% confusion of the confusable characters, line-break hyphenation of 10% of long words), **code-mixed** (function words replaced by Romanised regional words and regional particles inserted) and **both**. The noise is applied segment by segment, so the gold entity spans stay exact. The proportion of word tokens that the original transducer cannot analyse is the **out-of-lexicon rate**:

Table 63: Out-of-lexicon rate of the four test conditions

Condition	Word tokens	Out-of-lexicon rate
clean	4,125	1.9%
ocr	4,359	19.2%
mix	4,482	21.8%
both	4,691	36.0%

The code-mixed and the combined conditions exceed the 20% out-of-lexicon rate expected in the changed requirement (21.8% and 36.0%).

Hand-traces of three records through the original pipeline (the clean-corpus record of the same kind is processed without a problem):

Table 64: What fails first in three noisy records

Record (start of text)	Entities missed by the patterns (first failure)	Words with no analysis
OCR (R0036) `Ward l, Bed 32: patient fit, scrum dried for 6 hours. BP 122/91, temp 104.0 F, glucose 98 rng/dL. Complai…`	WARD “Ward l, Bed 32”, LAB “glucose 98 rng/dL”, DRUG “Enoxaparin 5m1”	l, scrum, F, rng, dL, suturd, condltions
code-mixed (R0061) `UHID 199270. 70-year-old stable patient admitted on 19/05/2024 to W2A/B21. vision OD 6/6. Presented oda k…`	none	year-old, W, B, oda, kuduthen, matrum, matrum
OCR + code-mixed (R0106) `MRN675791 39 y/o. Dx kuduthen A41.9 sacral steroi-\ndds. Amlodipine 50 mg TDS x 11 weeks. Follow up kudut…`	DRUG “Warfarin 5 rng”	y, o, kuduthen, steroi, dds, Amlodipine, x

Order of failure. In the OCR record (R0036) the first stage to fail is the **entity patterns**: WARD “Ward l, Bed 32”, LAB “glucose 98 rng/dL”, DRUG “Enoxaparin 5m1”; the transducer then fails on l, scrum, F, rng, dL, suturd. In the code-mixed record (R0061) the patterns miss none, so the first failure is the **transducer**: year-old, oda, kuduthen, matrum, matrum have no analysis, and the tokenizer leaves the fragments W and B from “W2A/B21”. In the combined record (R0106) the

failures add up: the patterns miss DRUG “Warfarin 5 rng”, and the transducer fails on y, o, kuduthen, steroi, dds, Amlodipine, including OCR-damaged lexicon words such as steroidds.

Measured degradation. The table compares the original pipeline on the clean, OCR, code-mixed and combined samples (entity precision and recall of the twelve patterns, analyser coverage of non-entity words, and retrieval F1 of the full-analysis system on the 40 queries).

Table 65: Degradation of the original design against the clean baseline

Condition	Entity P	Entity R	Analyser coverage	Retrieval P	Retrieval R	Retrieval F1
clean	0.884	0.962	96.9%	1.000	0.946	0.972
ocr	0.870	0.843	79.9%	1.000	0.810	0.895
mix	0.884	0.962	77.3%	1.000	0.946	0.972
both	0.876	0.857	63.3%	1.000	0.796	0.886

Entity recall falls from 96.2% to 84.3% under OCR noise; coverage falls from 96.9% to 63.3% in the combined case and retrieval F1 from 0.972 to 0.886. Precision is unaffected because unknown words simply produce no key: the system degrades by **silently missing** records, not by returning wrong ones. Code-mixing alone leaves retrieval almost unchanged, because the regional words replace function words and particles that no query asks for; its cost is the lower coverage and the lost analyses.

Would a different formalism be more economical? For three of the five failures yes. A **weighted edit-distance** match against the lexicon repairs OCR-damaged words without adding a single rule to the transducer (one confusion matrix covers every word); a **character n-gram language identifier** separates the regional tokens at a cost of about 60 word-list entries; and a **weighted automaton** would be the natural formal frame for both (arcs carrying confusion costs). Extending the existing transducer instead would need a rule for every confusion at every position, multiplying the lexicon several times. The one failure no formalism change repairs is **regional morphology**: understanding kuduthen as “gave” needs a regional lexicon.

C. Adaptation: MODIFY (supplement the finite-state layer)

The finite-state analyser stays as it is. Five components are added around it (Figure 19); they are shaded in the diagram.

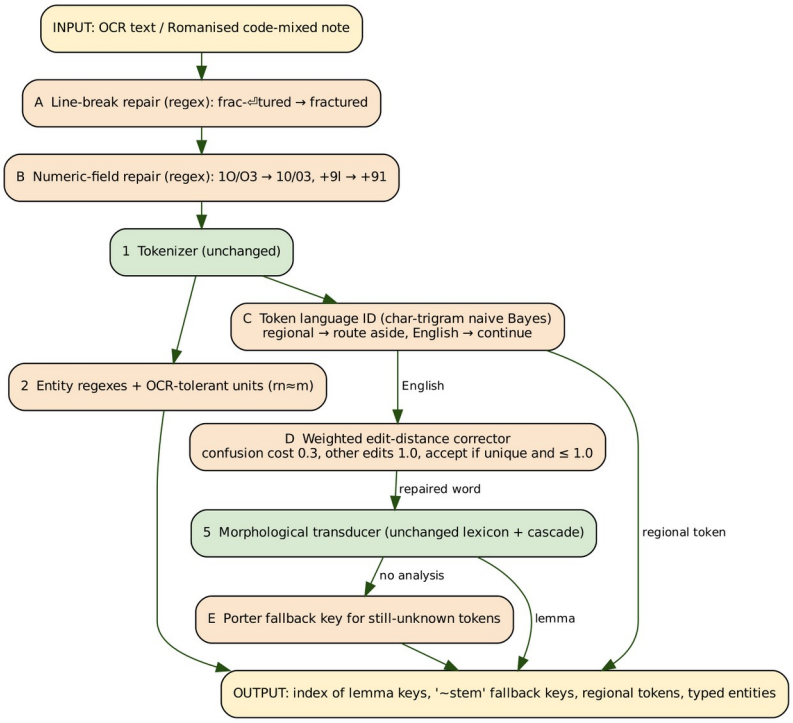


Figure 19: Revised pipeline for OCR and code-mixed input (shaded boxes are new)

Table 66: Exactly what changes in each part of the system

Part	Change	What stays
Patterns	Numeric-field repair before matching (O/o → 0, l/I → 1 inside digit runs); unit alternatives made tolerant (m|rn in mg, ml, mmHg, mmol/L)	All twelve patterns and their ranges
Automata	None; they now see repaired strings	The five DFAs, the ε-NFA
Lexicon	Add 62 Romanised regional words to a separate language-ID word list (not to the English FST); no regional morphology	463 English stems, all affixes
Rule cascade	None	Rules R1–R8
New: line-break repair	Regex joins "word-↵word" before tokenisation	–
New: language ID	Character-trigram naive Bayes (62 regional, 1,432 English training words); a token is regional if the log-odds margin exceeds 3 and it has ≥ 4 letters	–
New: corrector	Weighted edit distance: OCR confusions (o/0, l/1, c/e, i/l, rn/m …) cost 0.3, other edits 1.0; accept the unique best candidate within 1.0	–
New: fallback	A token still unknown gets the key "~" + Porter stem	The Porter stemmer

Effect. The same three samples were processed by the revised pipeline.

Table 67: Original against revised pipeline on the noisy samples

Condition	Pipeline	Entity recall	Analyser coverage	Retrieval P	Retrieval R	Retrieval F1	ms / record
ocr	original	0.843	79.9%	1.000	0.810	0.895	0.8
	revised	0.892	89.2%	0.993	0.932	0.961	3.1
mix	original	0.962	77.3%	1.000	0.946	0.972	0.4
	revised	0.949	77.9%	1.000	0.973	0.986	2.3
both	original	0.857	63.3%	1.000	0.796	0.886	0.5
	revised	0.899	71.5%	1.000	0.905	0.950	4.1

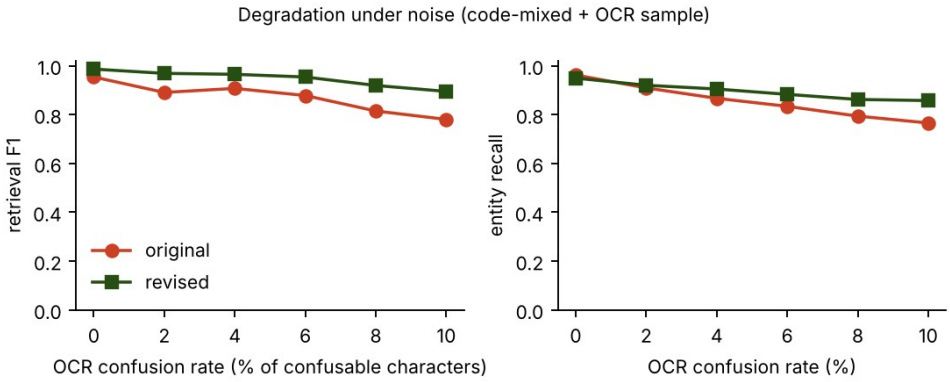


Figure 20: Original against revised pipeline as the OCR confusion rate rises

Quantified effect. On the combined sample the revised pipeline lifts retrieval F1 from 0.886 to 0.950 (recall 79.6% → 90.5%), entity recall from 85.7% to 89.9% and the analyser coverage of non-entity words from 63.3% to 71.5%. On the OCR sample alone, F1 rises from 0.895 to 0.961. The price is **run-time**: about 4.1 ms per record against 0.5 ms, a factor of 8, because every unknown token is compared with the candidate lexicon forms. It is also not free of side effects: on **clean** text the numeric repair costs entity recall (96.2% → 94.9%), because it turns an ICD code such as I10 into 110; the revised patterns should therefore be switched on only for sources that are marked as scanned. The language identifier reaches 64% recall on unseen regional words with 1.5% false alarms on English forms (5.8% on OCR-damaged English), so many short regional words (ku, oda) fall through to the Porter fallback, which is harmless.

Annotation effort to maintain the system. The new components need: the 62 regional words (about 1 hour), a confusion matrix of OCR errors (2 hours from a sample of scanned forms), tuning the thresholds on a development sample of 100 noisy records (3 hours), and, if regional morphology is to be analysed later, a regional lexicon and affix list comparable to the English one (about 40 hours). Estimated total for the modification: **about 6 hours**, against about 40 hours for a regional transducer.

D. Evidence-based defence of the decision

Why MODIFY is the right decision. The evidence points the same way in every experiment:

- **Retain would fail.** The original design loses 15.0% of recall and 33.6% of analyser coverage on the combined sample, and entity recall drops from 96.2% to 85.7%. The Task 2 error analysis already showed that patterns are fragile at the character level; the noise simply makes that fragility the dominant error.
- **Redesign is not justified.** The lexicon, rules and transducer are not the source of the loss: with the front end, coverage and F1 recover to 71.5% and 0.950. Replacing the analyser with a learned model would, by the Task 11 curve, cost about 33 hours of annotation to match what the lexicon already gives, and would lose the explainable lemma and feature output.
- **The front end is cheap.** Its estimated effort is about 6 hours, and it adds 4.1 ms per record, which is negligible for batch indexing of scanned forms.

What a finite-state layer will never handle, however far it is extended.

- **Meaning in context:** negation (“no fracture”), certainty (“?fracture”), and which patient or side a statement is about need syntax, semantics and discourse; the transducer never sees beyond the word.
- **Unbounded structure:** nested brackets and recursive notes are not regular (Task 4); a finite-state device cannot check them.
- **Open vocabulary:** new drugs, new regional words and new misspellings will keep arriving; a hand-built lexicon is always out of date, and the language identifier at 64% recall on unseen regional words shows where a learned model has to take over.
- **Genuine ambiguity:** “OD”, “left” and “leaves” need a probabilistic or contextual decision, which a transducer can only defer.

The right long-term architecture is therefore a **hybrid**: the finite-state layer for exact, explainable, cheap normalisation of structured and regular parts of the text, with a statistical component for ambiguity, noise and the regional language.

Residual failures after modification

Table 68: Entities still missed by the revised pipeline in the traced records

Record	Condition	Entities still missed
R0036	OCR	WARD “Ward l, Bed 32”, DRUG “Enoxaparin 5m1”
R0061	code-mixed	none
R0106	OCR + code-mixed	none

The residual misses come from characters the repair rules do not touch: an isolated l or I that stands for the digit 1 in a one-character field (“Ward l”), and the letter l in a unit read as the digit 1 (“5m1” for 5 ml). A better confusion model, trained on scanned forms, would address these; I list them instead of hiding them.

The Day-4 modules as code

```
def ocr_noise(token, r, p=0.05):
    out = []
    for ch in token:
        if ch in OCR_SUB and r.random() < p: out.append(OCR_SUB[ch])
        else: out.append(ch)
    t = ''.join(out)
    if len(t) >= 8 and token.isalpha() and r.random() < 0.10:      # word broken across a line
ending
        k = r.randint(3, len(t) - 3); t = t[:k] + '-\n' + t[k:]
    return t

def noisy_record(rec, r, ocr=True, mix=True, p=0.05):
    """returns (text, gold_spans): noise is applied segment-wise so gold entity spans stay exact."""
    segs = []
    for s, lab in rec.segs:
        if lab is None and mix and s.lower() in FUNC and r.random() < 0.9: s = FUNC[s.lower()]
        if ocr: s = ocr_noise(s, r, p) if (lab is not None or s not in '.,;:') else s
        segs.append((s, lab))
        if mix and lab is None and s.isalpha() and r.random() < 0.10:
segs.append((r.choice(PARTICLES),
             None))
    out = ''; spans = []
```

```
    for s, lab in segs:
        if out and s not in '.,;:': out += ' '
        st = len(out); out += s
        if lab: spans.append((lab, st, len(out)))
    return out, spans

def wed(a, b, cap=1.6):
    """Levenshtein with cheap OCR-confusion substitutions (0.3), and rn<->m, ri<->n as 0.3 two-for-
one
        moves."""
    n, m = len(a), len(b)
    D = [[0.0] * (m + 1) for _ in range(n + 1)]
    for i in range(1, n + 1): D[i][0] = i
    for j in range(1, m + 1): D[0][j] = j
    for i in range(1, n + 1):
        lo = 1e9
        for j in range(1, m + 1):
            sub = 0 if a[i - 1] == b[j - 1] else (0.3 if frozenset((a[i - 1], b[j - 1])) in CONF
else
                1.0)
            v = min(D[i - 1][j] + 1, D[i][j - 1] + 1, D[i - 1][j - 1] + sub)
            if i >= 2 and a[i - 2:i] == 'rn' and b[j - 1] == 'm': v = min(v, D[i - 2][j - 1] + 0.3)
            if i >= 2 and a[i - 2:i] == 'ri' and b[j - 1] == 'n': v = min(v, D[i - 2][j - 1] + 0.3)
            D[i][j] = v; lo = min(lo, v)
        if lo > cap: return cap + 1
    return D[n][m]

class Corrector:
    def __init__(self, surfaces):
        self.by_len = defaultdict(list)
        for w in surfaces: self.by_len[len(w)].append(w)
        self.surf = set(surfaces)
    def correct(self, tok, cap=1.0):
        if tok in self.surf or len(tok) < 4: return None
        best, bw = cap + 1, []
        for L in range(len(tok) - 2, len(tok) + 3):
            for w in self.by_len.get(L, ()):
                if w[0] != tok[0] and wed(tok[:1], w[:1]) >= 1: continue
                d = wed(tok, w, cap)
                if d < best: best, bw = d, [w]
                elif d == best: bw.append(w)
        return bw[0] if best <= cap and len(bw) == 1 else None

class LangID:
    def __init__(self, english, regional, n=3):
        self.n = n; self.c = {'en': Counter(), 'rg': Counter()}; self.tot = {'en': 0, 'rg': 0}
        for lab, ws in (('en', english), ('rg', regional)):
            for w in ws:
                for g in self.grams(w): self.c[lab][g] += 1; self.tot[lab] += 1
        self.V = len(set(self.c['en']) | set(self.c['rg']))
    def grams(self, w):
        w = '^' + w.lower() + '$'
        return [w[i:i + self.n] for i in range(max(1, len(w) - self.n + 1))]
    def predict(self, w, margin=3.0):
        if len(w) < 4: return 'en'
        sc = {}
        for lab in ('en', 'rg'):
            sc[lab] = sum(math.log((self.c[lab][g] + 0.5) / (self.tot[lab] + 0.5 * self.V)) for g in
                self.grams(w))
        return 'rg' if sc['rg'] - sc['en'] > margin else 'en'

def repair_numeric(text):
    """inside any run of [0-9OolI] that contains a digit, map O/o->0 and l/I->1 (not after the
keyword
        ICD)."""
    def fix(m):
        run = m.group(0)
        if not any(c.isdigit() for c in run): return run
        return run.translate(str.maketrans('OolI', '0011'))
    return re.sub(r'(?<![A-Za-z])[0-9OolI]{2,}(?![A-Za-z])|(?<=[-:/ ])[0-9OolI]+(?=[/.\- ])', fix,
text)

def dehyphenate(text):
    return re.sub(r'([A-Za-z]+)-\s*\n\s*([A-Za-z]+)', r'\1\2', text)

def tolerant(p):
    return _re.sub(r'mmHg|mmol/L|mg/dL|mcg|\bmg\b|mg(?=\|)|mg\)|ml\b|ml(?=\|)|ml\)', lambda m:
        _UNIT.get(m.group(0).rstrip('|)'), m.group(0)) + (m.group(0)[-1] if m.group(0)[-1] in '|)'
else
        ''), p)
```

MARK DISTRIBUTION

Task	**Marks**
Task 1 – Frame the Problem: Models, Algorithms and Levels of Analysis	3
Task 2 – Basic Regular Expression Patterns	4
Task 3 – Regular Expressions for Tokenisation and Normalisation	4
Task 4 – Finite State Automata: Design, Non-determinism and Equivalence	5
Task 5 – Inflectional Morphology	3
Task 6 – Derivational Morphology	4
Task 7 – Finite-State Morphological Parsing	5
Task 8 – Orthographic Rules and Two-Level Morphology	4
Task 9 – The Porter Stemmer	5
Task 10 – Implement and Test the Pipeline	3
Task 11 – Compare, Recommend and Defend the Normalisation Design	2
Subtotal	**42**
Task 12 – Day-4 Design Challenge	**8**
TOTAL	**50**